Ground Truth Inference for Weakly Supervised Entity Matching

RENZHI WU, Georgia Institute of Technology, USA
ALEXANDER BENDECK, Georgia Institute of Technology, USA
XU CHU, Georgia Institute of Technology, USA
YEYE HE, Microsoft Research, USA

Entity matching (EM) refers to the problem of identifying pairs of data records in one or more relational tables that refer to the same entity in the real world. Supervised machine learning (ML) models currently achieve state-of-the-art matching performance; however, they require a large number of labeled examples, which are often expensive or infeasible to obtain. This has inspired us to approach data labeling for EM using *weak supervision*. In particular, we use the labeling function abstraction popularized by Snorkel, where each labeling function (LF) is a user-provided program that can generate many noisy match/non-match labels quickly and cheaply. Given a set of user-written LFs, the quality of data labeling depends on a *labeling model* to accurately infer the ground-truth labels. In this work, we first propose a simple but powerful labeling model for general weak supervision tasks. Then, we tailor the labeling model specifically to the task of entity matching by considering the EM-specific transitivity property.

The general form of our labeling model is simple while substantially outperforming the best existing method across ten general weak supervision datasets. To tailor the labeling model for EM, we formulate an approach to ensure that the final predictions of the labeling model satisfy the transitivity property required in EM, utilizing an exact solution where possible and an ML-based approximation in remaining cases. On two single-table and nine two-table real-world EM datasets, we show that our labeling model results in a 9% higher F1 score on average than the best existing method. We also show that a deep learning EM end model (DeepMatcher) trained on labels generated from our weak supervision approach is comparable to an end model trained using tens of thousands of ground-truth labels, demonstrating that our approach can significantly reduce the labeling efforts required in EM.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity matching, weak supervision, labeling model, transitivity

ACM Reference Format:
Renzhi Wu, Alexander Bendeck, Xu Chu, and Yeye He. 2023. Ground Truth Inference for Weakly Supervised Entity Matching. *Proc. ACM Manag. Data* 1, 1, Article 32 (May 2023), 28 pages. https://doi.org/10.1145/3588712

1 INTRODUCTION

Entity matching (EM) refers to the process of determining if a pair of records from two data sources refer to the same real-world entity. EM has many applications, for example, in matching product listings for competitive pricing [2] and in building knowledge graphs [28]. As a long-standing problem, EM has been extensively studied (e.g., see surveys [18, 29, 30, 34, 36, 76]).

Authors' addresses: Renzhi Wu, Georgia Institute of Technology, Atlanta, USA, renzhiwu@gatech.edu; Alexander Bendeck, Georgia Institute of Technology, Atlanta, USA, abendeck3@gatech.edu; Xu Chu, Georgia Institute of Technology, Atlanta, USA, xu.chu@cc.gatech.edu; Yeye He, Microsoft Research, Redmond, USA, yeyehe@microsoft.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART32 $15.00
https://doi.org/10.1145/3588712

The Need for Weak Supervision in EM. The most important requirement for EM solutions is high matching quality as it directly affects downstream application performance. State-of-the-art EM solutions that report the highest matching quality [29, 50, 59, 60] are supervised machine learning (ML) approaches that train a binary classifier to predict the label (match or non-match) for any tuple pair. However, these models require large numbers of labeled tuple pairs, which are often not available or costly to obtain [29]. The high human cost in data labeling has become a main bottleneck in adopting high-quality EM solutions in practice.

To meet the needs of data-hungry supervised ML models, ML practitioners have increasingly turned to *weak supervision* methods, in which a larger volume of cheaply generated, but often noisier, labeled examples is used in lieu of hand-labeled examples. Different forms of weak supervision have been investigated, including the use of non-expert crowd workers [33], pretrained models [24], and rules/patterns/heuristics [75].

To unify different forms of supervision, the *data programming* paradigm [69] has been proposed, in which users write *labeling functions* (LFs) to programmatically label training data, rather than manually labeling each example by hand. Each LF is a small user-provided program (e.g., in Python) that leverages noisy signals in the data or domain knowledge to provide a label (or abstain) for an input example. In binary classification tasks such as EM, the output of each LF is +1 (positive class), -1 (negative class), or 0 (abstain). The data programming approach was first implemented in Snorkel [66], a weakly supervised data labeling system. Snorkel allows users to write LFs and then uses a generative model to combine all LFs to produce probabilistic labels. The data programming approach has been widely adopted in various ML tasks [31, 32, 53, 54, 73, 81, 85].

Example Labeling Functions. The LFs shown in Figure 1 are two user-written LFs developed using an existing tool [84] for matching electronic products [1]. The first LF, "name_overlap", encodes the intuition that matching products should have similar "name" attributes. Specifically, pairs of products with sufficiently high/low word overlap in their names are predicted as matches (+1)/non-matches (-1). When the amount of word overlap does not provide conclusive evidence either way, the LF abstains (0). The second LF, "size_unmatch", uses a regular expression to search the screen size in patterns like "... Samsung **40'** LCD ...". When the screen sizes of a pair of products are different, the LF predicts the pair to be non-match (-1); otherwise, the LF abstains (0).

```
@labeling_function
def name_overlap(tpl_l, tpl_r):
    x = lower(tpl_l.name)
    y = lower(tpl_r.name)
    x = set(splitBySpace(x))
    y = set(splitBySpace(y))
    itsect = x.intersection(y)
    max_len = max(len(x), len(y))
    score = len(itsect) / max_len
    if score < 0.1:
        return -1
    elif score > 0.8:
        return 1
    else:
        return 0
```

```
@labeling_function
def size_unmatch(tpl_l, tpl_r):
    l_name_des = str(tpl_l.name) + " " \
                 + str(tpl_l.description)
    r_name_des = str(tpl_r.name) + " " \
                 + str(tpl_r.description)
    x = re.findall(r"[\d\.]+\'", l_name_des)
    y = re.findall(r"[\d\.]+\'", r_name_des)
    if len(x) == 0 or len(y) == 0:
        return 0
    x, y = x[0], y[0]
    if x != y:
        return -1
    else:
        return 0
```

Fig. 1. Two user-written LFs for the abt-buy dataset [1].

The predictions of multiple LFs can then be used to construct a *labeling matrix* X, where each row corresponds to a tuple pair, and each column corresponds to predictions of one LF for all tuple pairs. After constructing the labeling matrix, a *labeling model* is used to consolidate the labeling function predictions into a final label for each tuple pair. Figure 2 shows one instance of a labeling matrix with votes from LFs (LF1, LF2, ...) and inferred labels from a naive labeling model (+1 for match, -1 for non-match) as well as the unknown ground truth (GT) labels for each tuple pair.

Given a set of user-written LFs, the quality of data labeling depends on the *labeling model* that infers the ground-truth labels from the LFs. In this paper, we first present an embarrassingly simple

	LF1	LF2	LF3	...	Naïve Inferred Label	GT
(t_1, t_2)	1	1	1	...	1	1
(t_1, t_3)	1	1	-1	...	1	-1
(t_1, t_4)	1	1	0	...	1	1
...	...	...	...	...	...	...
(t_2, t_3)	0	-1	-1	...	-1	-1
(t_2, t_4)	0	0	-1	...	-1	1

Fig. 2. Example of a labeling matrix

yet highly effective labeling model (SIMPLE) for general weak supervision tasks. We then specialize the labeling model (SIMPLE-EM) for the task of EM by considering the transitivity property.

(1) Labeling Model for General Weak Supervision Tasks: Existing methods are mostly generative models, i.e., they model the process of generating the predictions of each LF from the hidden ground-truth labels [26, 32, 52, 68]. Under an assumed generative process, one could measure the likelihood of observing all LFs (i.e., the labeling matrix) and an estimation of the ground-truth labels can be obtained by maximizing the likelihood [26, 32, 52, 68]. A fundamental limitation of these approaches is the need to assume that the generative process takes certain simplified forms for ease of modeling or to make the model mathematically solvable. Such assumptions include the Markov assumption [26, 32, 68] and the mixture of independent sub-types assumption [52]. Evidently, these assumptions can often break in practice. Recent benchmark studies [86] suggest that the best existing labeling model for general weak supervision tasks is only 1.5% better than majority vote, and the second best method is already worse than majority vote. In addition, even with these various assumptions to simplify the generative process, the existing approaches are typically quite complicated and difficult to implement.

We take a fundamentally different approach that is a significant departure from the conventional wisdom that generative models should be used in labeling models. The core intuition of our method is that *labeling models are functions*. Conceptually, every labeling model takes in the labeling matrix X and predicts the label vector $\hat{\boldsymbol{y}}$. For example, X could be the sub-matrix in Figure 2 with columns of all LFs and $\hat{\boldsymbol{y}}$ could be the naive inferred label column in Figure 2. Designing a labeling model is to design a function G parameterized by Θ, and $\hat{\boldsymbol{y}} = G(X, \Theta)$ would be the predicted labels of the model. The prediction of the whole matrix X can be made by independently predicting each individual row/data point; i.e., $\hat{y_i} = g(X[i, :], \theta)$ where θ is the parameter of g. Different designs of existing labeling models can be seen as the function g taking different forms.

As discussed, existing labeling models typically build complicated models (e.g., probabilistic graphical models [27, 41, 55] and matrix completion models [38, 67]) by making various assumptions to handcraft g. In this paper, we ask a fundamental question that is contrary to the conventional wisdom: *Can we use a generic classifier (e.g., random forest) as g to be the labeling model and avoid all the complicated designs in the existing approaches?* We provide an affirmative answer to this question by presenting an embarrassingly simple method (in Section 3) that achieves better performance than the existing approaches across ten datasets.

(2) EM-specific Labeling Model with Transitivity: The *transitivity property of EM* states that for any three tuples t_i, t_j, and t_k, if t_i matches t_j and also matches t_k, then t_j has to match t_k. Concretely, consider the scenario in Figure 2 – a naive model would likely predict the tuple pair (t_2, t_4) as non-match, despite strong evidence that both (t_1, t_2) and (t_1, t_4) should be matches, which results in inconsistent label assignments that violate transitivity. Our goal is to explicitly model transitivity using constraints, so that matching decisions can be made in a holistic manner.

A naive approach to enforce transitivity is to do a post-processing step, which is sub-optimal. Intuitively, the additional signals from the transitivity constraints should be made an integral part of the labeling model to improve its accuracy. We thus attempt to incorporate transitivity directly in the labeling model. However, this makes the labeling model complex and difficult to solve.

To tackle this challenge, we first consider a simplified yet common scenario in two-table EM, where at least one table is "duplicate-free" (this has been shown to hold for many real-world datasets [48] which we further verify in Section 5.5). In this scenario, we can derive an exact solution incorporating transitivity in the labeling model. In the more general setting where neither table is duplicate-free or in single-table EM, we propose an ML-based approach (in Section 4) that produces an approximate solution with transitivity incorporated into the labeling model. We highlight that our ML model can be trained offline once and then used for any new EM datasets without needing any form of update.

Contributions. In this work, we make the following contributions:

(1) A generic view of labeling models. We present a generic view of labeling models as functions (or classifiers). Different existing methods are simply various instantiations of this generic view.

(2) A simple and powerful labeling model. Based on the generic view, we study a fundamental question: Is it possible to use a generic classifier as the labeling model? We provide an affirmative answer by proposing a simple yet powerful labeling model based on random forest. Our method achieves better results than existing approaches across ten general weak supervision datasets.

(3) Exact solution for transitivity. We adapt our proposed labeling model to the task of two-table entity matching by considering the transitivity property. We present an exact solution to transitivity when at least one table is duplicate-free for two-table EM.

(4) ML-based solution for transitivity. We adapt our proposed labeling model to the task of single-table entity matching by considering the transitivity property. We propose an ML-based approach based on the observation that, in an optimization step of the labeling model, the optimal matching probabilities under the transitivity constraint depend only on the optimal probabilities without considering the transitivity constraint. We further design a novel architecture for the ML model by considering task-specific properties. Our ML model trained once offline can be applied to any new EM datasets without needing any adaptation.

2 PROBLEM DEFINITION

Given m labeling functions (LFs) and n data points to be labeled, the *labeling matrix* $X \in \{+1, -1, 0\}^{n \times m}$ is obtained by applying all LFs to all data points. Since LFs are noisy and may have dependencies (leading to overlaps and conflicts in X), our goal is to design and implement a *labeling model* to optimally combine the LF labels in X to obtain accurate final labeling results.

For the task of entity matching, each data point is a tuple pair (t_i, t_j) and the predicted label is denoted as $\hat{y}^{(i,j)} \in \{-1, +1\}$. The final predictions must satisfy the transitivity constraint that if $\hat{y}^{(i,j)}$ and $\hat{y}^{(i,k)}$ are both equal to +1 (denoting matches), then $\hat{y}^{(j,k)}$ must also be +1. This form of the transitivity constraint applies to single-table EM; for two-table EM, the matching probabilities of tuple pairs from the same table are typically not available, so in practice transitivity relies on one or both tables being duplicate-free.

3 PROPOSED LABELING MODEL

In this section, we introduce the general form of our proposed labeling model for general weak supervision tasks.

3.1 A Generic View of Labeling Models

Every labeling model (also referred to as truth inference method) takes in the labeling matrix X and predicts the label vector $\hat{\boldsymbol{y}}$. At a high level, we can think of a truth inference algorithm as a function G parameterized by Θ such that $\hat{\boldsymbol{y}} = G(X, \Theta)$. In most tasks, the data points are independent, so the predictions of their labels are also made independently. Therefore, the function G that makes

predictions for the whole labeling matrix can be expressed with a function g that makes a prediction for each individual data point. When predicting the ith data point, the function g takes in the ith weak label vector (i.e., the ith row in matrix X) $x_i = X[i, :]$ as features and predicts the probability of being in the positive class γ_i, i.e. $g(x_i, \theta)$ where θ is the parameter of g. The label $\hat{y}_i$ is obtained as $\hat{y}_i = 1$ if $g(x_i, \theta) \geq 0.5$ and $\hat{y}_i = 0$ otherwise. In this formulation, g is essentially a classifier that takes in a feature vector x_i and predicts a soft label. In principle, g could be any classifier, and the challenge is how to learn its parameter θ without labeled data.

Expectation-Maximization Algorithm. Most labeling models (or truth inference methods) adopt the Expectation-Maximization algorithm (or extensions of it) to learn the model parameter θ. The objective function is the negative log data likelihood function:

$$L(\theta, X, \boldsymbol{\gamma}) = -\Sigma_{i=1}^{N}\big(\gamma_i \log(g(x_i, \theta)) + (1 - \gamma_i)\log(1 - g(x_i, \theta))\big) \tag{1}$$

where $\boldsymbol{\gamma} = \{\gamma_1, \dots\}$ is the hidden ground-truth label. Note we use $\boldsymbol{\gamma}$ to denote the soft labels (the matching probabilities) and $\hat{\boldsymbol{y}}$ to denote the hard labels. Minimizing the negative log data likelihood function translates to maximizing the likelihood of observing the labeling matrix X. Since both the parameter θ and label $\boldsymbol{\gamma}$ are unknown, the Expectation-Maximization algorithm iteratively estimates θ by minimizing the objective function with $\boldsymbol{\gamma}$ fixed as the current estimation and then computes $\boldsymbol{\gamma}$ using the current estimated θ. Specifically, the learning workflow of Expectation-Maximization algorithm is as follows:

(1) Obtain an initial estimation of the hidden ground-truth label $\boldsymbol{\gamma}$ by majority vote, as this works well in practice [86].
(2) *M-step*: Estimate the model parameter θ by minimizing Equation 1 with respect to θ while keeping the labels $\boldsymbol{\gamma}$ fixed as the current estimation. This uses the current estimated labels to learn the model parameters.
(3) *E-step*: Update the estimated labels $\boldsymbol{\gamma}$ as the predicted matching probabilities $\gamma_i = g(x_i, \theta),\ \forall i$ using the model parameter θ obtained in the M-step.
(4) Repeat steps (2) and (3) until convergence.

It can be easily shown (by taking derivative and setting it equal to zero) that in the M-step, the global minimum of Equation 1 with respect to θ is achieved when $g(x_i, \theta) = \gamma_i$. This means the objective function measures the difference/loss between the model prediction and the current soft label, and thus one can replace the objective function in M-step to be any loss function $D(\gamma_i, g(x_i, \theta))$ which also has a minimum at $g(x_i, \theta) = \gamma_i$ to achieve similar results. In other words, in the M-step, we can update the model parameter θ by minimizing $\frac{1}{N}\sum_{i=1}^{N} D(\gamma_i, g(x_i, \theta))$ where N is the number of examples. Since some classifiers require specific forms for their loss functions (e.g., SVM and random forest), the possibility to substitute the objective function with any loss function while achieving similar results in the M-step enables us to instantiate g to be any classifier.

Instantiation as Different Labeling Models. By choosing different forms of $g(x_i, \theta)$, the Expectation-Maximization algorithm is instantiated as different existing labeling models. The existing methods design $g(x_i, \theta)$ by making different assumptions.

One common assumption is that the noisy labels of each LF are only dependent on the ground-truth labels, so the noise can be modeled by the joint probability table (the confusion matrix) of the LF and the hidden ground-truth. Based on this, a family of truth inference methods [26, 70, 79] has been developed. These models can be obtained by instantiating $g(x_i, \theta)$ as the probability distribution of the class labels derived based on the confusion matrix.

Another common assumption is the Markov assumption. This is the core assumption in probabilistic graphical models (PGM) [42]. With this assumption, the truth inference problem can be formulated as a PGM with hidden variables. A rich family of truth inference methods has been

developed using PGMs. Such models can be obtained if we represent $g(x_i, \theta)$ with a PGM. Different methods in this family differentiate from each other by adopting different priors [27], or by replacing the Expectation-Maximization algorithm with its extension (Variational Bayes [20]) [41, 55].

3.2 The SIMPLE Algorithm.

In principle, g can be any classifier. Our intuition for choosing a proper g is that the capacity (also referred to as complexity or size of the hypothesis space in the literature) of g should not be too large, otherwise g will learn a trivial solution (the prediction of majority vote) in the M-step of the first iteration. On the other hand, the capacity of g should not be too small as we need g to capture the interaction and dependency of different LFs (features).

In fact, different existing methods can be viewed as restricting the capacity of g in different ways through different intuitions or assumptions. Specifically, by designing g based on the assumptions (e.g., conditional independence), these methods implicitly restrict the space of g to only include functions that observe the assumptions, so the model avoids learning the trivial solution (the initialized labels, e.g., by majority vote) and can generalize well.

From this view, one natural question is the following: Is it really necessary to handcraft complicated models with various assumptions to limit capacity? Or can we directly use a generic classifier as g and explicitly limit its capacity to achieve similar results? The answer to this latter question turns out to be yes. We show that we can actually use a generic classifier (random forest) as g and achieve better performance by explicitly restricting its capacity.

To use a generic classifier as g, a straightforward choice is to use a simple linear classifier (i.e., logistic regression). This reduces to the weighted majority vote method. In other words, logistic regression assigns a weight for each LF and then combines all LFs using the weights to get the final label. The limitation of logistic regression is that it is not able to capture more complex interactions or dependencies between different features (LFs).

To choose a proper classifier, our intuition is that we want a classifier (1) that is able to express the interaction of different LFs, and (2) for which we can easily restrict its capacity. Tree-based approaches naturally model the interaction of different features (LFs). For example, in a decision tree, the model makes decisions based on different features at different levels of the tree, so the tree as a whole naturally considers the interactions of different features (LFs). Tree-based approaches also work very well on structured data in practice. In fact, a recent survey reveals that tree-based methods are still the most common winning solutions in ML competitions on structured data [63]. Therefore, we choose the random forest classifier, a classic tree-based method, for our task. Note that we can easily restrict the capacity of a random forest classifier by setting its hyper-parameters. The first parameter is the maximum tree depth $d_{\max}$. With a smaller maximum depth, the random forest classifier has smaller capacity. Another parameter that controls the overall capacity/complexity of the trees is the complexity parameter: ccp_alpha. Following the common practice, we select both parameters $d_{\max}$ and ccp_alpha using cross validation. Note cross validation is done with the current estimated labels at each M-step where we train the classifier and no ground-truth labels are used.

Restricting the capacity of a classifier is also known as regularization. In our method, regularization is done as in typical ML tasks – in a way that is explicit and data-driven through cross validation. In contrast, in existing truth inference methods, regularization is done implicitly through manually restricting the hypothesis space (e.g., the form of function g) based on various assumptions or heuristics.

With a random forest classifier as g, the M-step in the Expectation-Maximization algorithm simply becomes training the classifier with the current estimated labels (we use the hard labels $\hat{\boldsymbol{y}}$ obtained by binarizing the soft labels $\boldsymbol{\gamma}$ because common implementations of random forest only

support training with hard labels), and the E-step simply becomes performing prediction on the training set to obtain an updated version of the soft labels $\boldsymbol{\gamma}$.

Class Imbalance. Many real world datasets (especially EM datasets) have imbalanced classes. Traditionally, to handle the data imbalance problem, one would need to carefully design the labeling model (truth inference method) such as by introducing priors [32, 52, 66]. In our method, handling the problem of class imbalance is the same as handling class imbalance in a typical ML setting, and we are able to directly adopt state-of-the-art techniques to address it.

We handle the class imbalance problem when training the classifier at the M-step. Specifically, we augment the data points of the minority class to match the size of the majority class with SMOTE [17], a simple technique for class imbalance that works very well in practice. SMOTE [17] works by creating synthetic examples by interpolating existing data points. For example, if we have two data points in the positive class $(x_1, 1)$, $(x_2, 1)$, SMOTE might create one synthetic data point by interpolating the two data points, e.g., $(\frac{x_1+x_2}{2}, 1)$. We train the model with the augmented minority class and the original majority class at each M-step. After training, in the E-step, we perform prediction on the original set of data points to get a new version of the labels $\boldsymbol{\gamma}$. Since we do not know whether a given dataset has the class imbalance problem or not, we always apply SMOTE at each M-step for all datasets. The pseudo-code of the entire SIMPLE algorithm is shown in Algorithm 1. We open-source our implementation at [8].

Algorithm 1: SIMPLE

Input: Labeling matrix X
Output: Estimated soft labels $\boldsymbol{\gamma}$

$\boldsymbol{\gamma} \leftarrow$ majority vote on X
while *Not Converged* **do**
  M Step
    Obtain hard labels $\hat{\boldsymbol{y}}$ by binarilize the soft labels $\boldsymbol{\gamma}$.
    Make the classes balanced: $X', \hat{\boldsymbol{y}}' = \text{SMOTE}(X, \hat{\boldsymbol{y}})$
    Select random forest parameters $d_{\max}$ and ccp_alpha with cross validation on data $(X', \hat{\boldsymbol{y}}')$
    RandomForestClassifier.fit$(X', \hat{\boldsymbol{y}}')$
  E Step
    $\boldsymbol{\gamma} \leftarrow$ RandomForestClassifier.predict_proba(X)
end
return $\boldsymbol{\gamma}$

Computational complexity. The complexity of each iteration is dominated by training the random forest classifier, which has a time complexity of $O(N \log(N))$ where N is the number of tuple pairs in the candidate set [56]. Let M_I denote the number of iterations. The overall time complexity is $O(M_I N \log(N))$. In our experiments, we observe that 10 iterations is enough for all datasets. The space complexity is $O(N)$.

Discussion. Our method has a connection to the pseudo-labeling method in semi-supervised learning [14, 47]. In pseudo-labeling, first a model is trained on a labeled training set and used to make predictions on unlabeled data; the data with confident predicted labels are then added to the training set. Next, the model is trained on the new training set, and then this process continues iteratively. Both pseudo-labeling and our method use predicted data as labeled data for the next iteration. The differences between pseudo-labeling and our method are as the following: First,

pseudo-labeling is in a semi-supervised setting while our method is in a unsupervised setting. We highlight that truth inference might be the only (or at least one of the few) unsupervised task(s) where one could apply this type of iterative approach as one can get a reasonable initial estimation (e.g., by majority vote in truth inference) to start with. This is due to the implicit assumption in weak supervision that each user-provided LF is better than random guessing. For general unsupervised ML tasks, there is no straightforward way to get a good initial estimation. Second, the focus of pseudo-labeling is to train a model while our focus is to obtain the labels for all data points; consequently, pseudo-labeling only adds confident label predictions to the training set while we include all predictions.

4 INCORPORATING TRANSITIVITY

In this section, we tailor our proposed labeling model to the task of EM by incorporating the transitivity property. The tailored method is denoted as SIMPLE-EM. The transitivity property for EM states that if tuple t_i matches t_j and t_i matches t_k, we must conclude that t_j also matches t_k. We follow prior work [80] to model transitivity as an inequality constraint, and the E-step of the EM algorithm can then be formulated as a constrained optimization problem. To make this section self-contained, we briefly review the formulation from prior work [80] in Section 4.1. Then, we introduce our solution in Section 4.2 and Section 4.3.

4.1 Constrained Optimization Formulation for Transitivity.

Transitivity as Constraint on Matching Probabilities. We follow prior work [80] to model transitivity as an inequality constraint defined on matching probabilities. Specifically, for any three tuples t_i, t_j, and t_k, the transitivity constraint can be expressed as the following inequality:

$$\gamma^{(i,j)} \times \gamma^{(i,k)} \leq \gamma^{(j,k)} \tag{2}$$

where we use the superscript (i, j) (e.g., $\gamma^{(i,j)}$) to index the tuple pair (t_i, t_j) and $\gamma^{(i,j)}$ denotes the matching probability of (t_i, t_j). To see how Equation 2 secures the transitivity constraint, consider an example where $\gamma^{(i,j)} = 0.6$ and $\gamma^{(i,k)} = 0.5$: in 60% of the cases (t_i, t_j) is a match and in 50% of the cases (t_i, t_k) is a match. Transitivity applies only when both (t_i, t_j) and (t_i, t_k) are matches which is in $60\% \times 50\% = 30\%$ of the cases, and in these cases (t_j, t_k) is a match due to transitivity. Therefore, (t_j, t_k) has at least a 30% chance of being a match, as captured by Equation 2.

The set of all transitivity constraints defines a *feasibility set* for $\boldsymbol{\gamma}$: $Q = \{\boldsymbol{\gamma} | \gamma^{(i,j)}\gamma^{(i,k)} \leq \gamma^{(j,k)} \forall i, j, k\}$. Intuitively, incorporating transitivity reduces to ensuring that the labeling model's prediction $\boldsymbol{\gamma}$ is in the feasibility set Q.

Incorporating Constraints in EM. On the surface, it seems very difficult to incorporate the transitivity constraints Q into the labeling model – namely, to find the best model parameter θ that minimizes $L(\theta, X, \boldsymbol{\gamma})$, while ensuring that the probabilities $\boldsymbol{\gamma}$ that are directly computed based on θ in the E-step satisfy Q. Following prior work [80], this is solved by using the free energy view of the expectation-maximization algorithm [62]. In this view, the objective function becomes the negative free energy function $F(\theta, X, \boldsymbol{\gamma})$:

$$F(\theta, X, \boldsymbol{\gamma}) = \sum_{(i,j)} -\gamma^{(i,j)} \log \frac{g(x^{(i,j)}, \theta)}{\gamma^{(i,j)}} - (1 - \gamma^{(i,j)}) \log \frac{1 - g(x^{(i,j)}, \theta)}{1 - \gamma^{(i,j)}} \tag{3}$$

The M-step stays the same as it was before, while the E-step also becomes an optimization process that allows us to incorporate the transitivity constraints. Specifically, the E-step becomes $\boldsymbol{\gamma}^* = \arg\min_{\boldsymbol{\gamma}} F(\theta, X, \boldsymbol{\gamma})$ which can be shown to be equivalent to the original E-step where $\boldsymbol{\gamma}^*$ is obtained by direct computation (i.e., $\gamma^{*(i,j)} = g(x^{(i,j)}, \theta)$) [62]. With this formulation, the transitivity

constraint can be incorporated in E-step by constraining $\boldsymbol{\gamma} \in Q$:

$$\boldsymbol{\gamma}^{**} = \arg\min_{\boldsymbol{\gamma} \in Q} F(\theta, X, \boldsymbol{\gamma}) \tag{4}$$

Intuitively, $\boldsymbol{\gamma}^{**}$ would be the best matching probabilities that satisfy the transitivity constraint while minimizing the objective function. However, the above constrained optimization problem is difficult to solve since the constraint set Q is actually non-convex (the Hessian matrix of a constraint is indefinite) [80]. Existing work uses a projection-based heuristic [80] to address this, which is not robust across datasets as we show in experiments. We will propose a more principled and efficient solution.

Relationship Between Constrained and Unconstrained Solution. The observation of our solution is that the constrained solution $\boldsymbol{\gamma}^{**}$ is only dependent on the unconstrained solution $\boldsymbol{\gamma}^{*}$; in other words, there exists a function h such that $\boldsymbol{\gamma}^{**} = h(\boldsymbol{\gamma}^{*})$. To see this, $\boldsymbol{\gamma}^{*}$ is obtained as $\gamma^{*(i,j)} = g(x^{(i,j)}, \theta)$ in the E-step and we can replace $g(x^{(i,j)}, \theta)$ with $\gamma^{*(i,j)}$ in the objective function in Equation 3. In this way, the only two variables in the objective function are $\gamma^{*(i,j)}$ (which is known) and $\gamma^{(i,j)}$ (which is to be solved and the solution is denoted as $\gamma^{**(i,j)}$). Since $\boldsymbol{\gamma}^{*}$ and $\boldsymbol{\gamma}$ are the only two variables in the objective function and the constraint of Equation 4, the optimal solution of $\boldsymbol{\gamma}$ (i.e., $\boldsymbol{\gamma}^{**}$) is only dependent on $\boldsymbol{\gamma}^{*}$.

4.2 Transitivity for Two-Table EM

For two-table EM, it has been found that in most real-world datasets at least one of the two tables is duplicate-free [48]. Under the scenario that one or both tables are duplicate-free, we can derive the exact $\boldsymbol{\gamma}^{**}$ from $\boldsymbol{\gamma}^{*}$. (Again, note that $\boldsymbol{\gamma}^{*}$ can be easily obtained by direct computation in the E-step.)

One Table Is Duplicate-free. Assume without loss of generality that the left table is known to be duplicate-free. For any tuple pair (t_{l_i}, t_{l_j}) from the left table, the duplicate-free information is incorporated into the model formulation by setting $\gamma^{**(l_i,l_j)} = \gamma^{*(l_i,l_j)} = 0$. We can show that in the constrained solution $\boldsymbol{\gamma}^{**}$, for any tuple t_{r_k} from the right table, there exists only one tuple from the left table that has non-zero matching probability with t_{r_k}. To see this by contradiction, let t_{l_i} and t_{l_j} denote two tuples from the left table that have non-zero matching probability with t_{r_k}. By the transitivity constraint, $\gamma^{**(r_k,l_i)}\gamma^{**(r_k,l_j)} \leq \gamma^{**(l_i,l_j)} = 0$, which means at least one of $\gamma^{**(r_k,l_i)}$ and $\gamma^{**(r_k,l_j)}$ must be zero, contradicting the fact that t_{l_i} and t_{l_j} both have non-zero matching probability with t_{r_k}.

Assume that we have the optimal unconstrained matching probabilities $\gamma_k^{*} = \{\gamma^{*(r_k,l_1)}, \gamma^{*(r_k,l_2)}, \dots\}$ between t_{r_k} and the left table tuples. Since the constrained solution can only have one of these probabilities be nonzero, the optimal solution under the transitivity constraint γ_k^{**} is obtained by setting $|\gamma_k^{*}| - 1$ values to be zero in γ_k^{*}. We would like to keep as nonzero the variable with the maximum increase in the objective function when it is set to zero, so that the overall objective is minimized. The objective function increase of a single probability $\gamma^{*(r_k,l_i)}$ is obtained as $\Delta F(\gamma^{*(r_k,l_i)}) = \log(1/(1-\gamma^{*(r_k,l_i)}))$ which is monotonic to $\gamma^{*(r_k,l_i)}$. As a result, the algorithm of obtaining $\boldsymbol{\gamma}^{**}$ from $\boldsymbol{\gamma}^{*}$ is as follows: For every tuple in the right table, find the left tuple with the maximum probability of matching to it, and set the matching probabilities of all other left tuples to be zero. The time complexity of this step is $O(N)$ and the time complexity of the overall SIMPLE-EM algorithm is still $O(M_I N \log(N))$. Note this method is the same as the one used in SiGMa [46] and LINDA [16]. In prior work [16, 46] the method was used as a greedy solution for the general setting, while we point out that in our formulation it is the optimal method in the case when one table is duplicate-free. When two tables are both duplicate-free, this method is sub-optimal/greedy and we propose a different solution in the following paragraph.

Two Tables Are Duplicate-free. When both tables (L table and R table) are known to be duplicate free, we can follow the same reasoning as in the one-table duplicate-free case and extend it to

be bi-directional. Thus, every tuple in the left table can have non-zero matching probability to only one tuple in the right table, and every tuple in the right table can have non-zero matching probability to only one tuple in the left table. Therefore, for all $|L| \times |R|$ possible left tuple and right tuple pairs, we want to keep $\min(|L|, |R|)$ pairs and set the matching probabilities of all other pairs to be zero. In particular, we would like to keep the $\min(|L|, |R|)$ pairs with the minimum objective function values while satisfying the condition that every left (or right) tuple can have non-zero matching probability to at most one tuple in the right (or left) table. This is essentially the *assignment problem*, and there is an existing efficient algorithm to solve it – the LAPJV algorithm [22, 40] with a time complexity of $O(N \min(N_l, N_r))$. Note that the time complexity listed in the original paper [40] is the dense version which corresponds to the setting without blocking. The complexity $O(N \min(N_l, N_r))$ is the sparse case with blocking and can be derived following [22]. Since we only care about matches, we only need to consider the pairs with a matching probability greater than 0.5. In this case, the time complexity can be further optimized to be $O(N_M \min(N_{l,M}, N_{r,M}))$, where N_M is the number of predicted matches (typically orders of magnitude smaller than N) and $N_{l,M}$ and $N_{r,M}$ are the number of left and right tuples involved in the predicted matches. In our experiments, we do not adopt this optimization as we empirically observed that the algorithm finishes in a reasonable time without the optimization. The time complexity of the SIMPLE-EM algorithm in this case is $O(M_I N(\log(N) + \min(N_l, N_r))$. The space complexity is $O(N)$. We use an existing efficient implementation of the LAPJV algorithm in the scipy package [6].

Since the LAPJV algorithm is non-trivial and technically dense, here we only briefly introduce the high-level ideas of the algorithm. The LAPJV algorithm first reformulates the assignment problem as a minimum cost flow problem and then solves it by finding the shortest path on an auxiliary graph [40]. It further employs several techniques (e.g., column reduction, reduction transfer, and augmenting row reduction) to quickly filter out unlikely paths [40]. For more details, one can refer to the original paper [40].

Duplicate-free Detection with Weak Supervision. We use the exact solutions outlined above when it is known one or two tables is duplicate-free. However, in some cases, the information of whether one table is duplicate-free is unknown. Therefore, we propose a method that leverages the results of the labeling model (without considering transitivity) that labels left-right (LR) tuple pairs with LFs to detect whether either table is duplicate free. The intuition of our detection method is that, if the left table is not duplicate-free, one right-table tuple might appear in multiple LR matching tuple pairs. For example, (t_{l_1}, t_{r_1}) and (t_{l_2}, t_{r_1}) are both matches; due to the fact the left table contains duplicates t_{l_1} and t_{l_2}, the right tuple t_{r_1} appeared twice in the matching pairs. On the other hand, when the left table is duplicate-free, one right-table tuple can only appear once in the LR matching tuple pairs. This means the distribution of the right-table tuples in the LR matching tuple pairs is different in the two cases. The predicted LR matching tuples pairs from the labeling model might be noisy, but still provide some information that we can use to detect which of the two cases the distribution of the right-table tuples falls in by using a hypothesis test procedure. We provide detailed description and experimental evaluation of the method in a technical report [7] due to space considerations. We highlight that the proposed duplicate-free detection method does not require LFs for left-left (LL) or right-right (RR) tuple pairs. Since the user has already written LFs for LR pairs, the method requires no additional effort from the user.

4.3 Transitivity for Single-Table EM

For single-table EM, we are not able to leverage the duplicate-free information to derive the exact constrained solution $\boldsymbol{\gamma}^{**}$ from the unconstrained solution $\boldsymbol{\gamma}^{*}$. Since we showed that $\boldsymbol{\gamma}^{**} = h(\boldsymbol{\gamma}^{*})$ at the end of Section 4.1, we propose to train a model offline to approximate h. Specifically, we randomly generate many instances of $\boldsymbol{\gamma}^{*}$, and employ expensive numerical solvers to obtain the

corresponding $\boldsymbol{\gamma}^{**}$. In this way, we obtain many pairs of $(\boldsymbol{\gamma}^{*}, \boldsymbol{\gamma}^{**})$, which are used as training data to train an ML model to approximate h.

Trained model will be dataset-agnostic. Before we go into the details of how we train the model, we emphasize that the trained model will also work on unseen datasets that may differ from the generated training set. To see this, consider that the form of the function h is dataset-independent as no dataset-specific information is involved in our derivation at the end of Section 4.1. If we can derive the analytical form of h, the analytical form can surely be used for any dataset. However, obtaining the analytical form is difficult. Intuitively, we could obtain all possible values in the domain of $\boldsymbol{\gamma}^{*}$ and numerically solve $\boldsymbol{\gamma}^{**}$ for each value and then save the result in a dictionary. In this way, we obtain a numerical representation of the function h as a dictionary. Then at inference time, for each value $\boldsymbol{\gamma}^{*}$ we could find the corresponding $\boldsymbol{\gamma}^{**}$ in the dictionary. Intuitively, the dictionary can be used for any unseen dataset as it is simply a different but equivalent representation to the analytical form. Our ML-based solution can be seen as a less expensive approximation of such a dictionary-based approach. Instead of obtaining all possible values in the input domain, we pick a subset of random values (i.e., the training set), and instead of saving every pair of one-to-one mapping between $\boldsymbol{\gamma}^{*}$ and $\boldsymbol{\gamma}^{**}$, we compress the mapping by using an ML model. Similar to the dictionary-based approach, the trained ML model can be used for any dataset. The way of learning an ML model to approximate functions (that are difficult to solve analytically) has also been adopted in other tasks recently [12, 37, 81, 83]. In these tasks, similar to ours, the trained model can be used for any unseen dataset [12, 37, 81, 83].

To manifest this idea, one challenge is that ML models require their inputs and outputs to have fixed dimensions, while the dimension of $\boldsymbol{\gamma}^{*}$ depends on the number of tuple pairs, which varies for different EM tasks. Our approach is to first train a model to approximate h for fixed-dimension $\boldsymbol{\gamma}^{*}$ (e.g., a 1024-dimensional $\boldsymbol{\gamma}$ is sufficient to represent at most 32 tuple pairs as there are $32 \times 32 = 1024$ matching probabilities). We then carefully decompose $\boldsymbol{\gamma}^{*}$ to sub-components with a maximum size of 1024 (i.e., clusters with 32 tuple pairs) and apply the model to each sub-component.

Training Data Generation. We randomly generate 10^5 matching probability matrices of size 32×32, each corresponding to a 1024-dimensional vector $\boldsymbol{\gamma}^{*}$. Each probability matrix corresponds to one training data point and we empirically observed that increasing the amount of training data from 10^5 to 10^6 does not bring meaningful improvement. For each $\boldsymbol{\gamma}^{*}$, $\boldsymbol{\gamma}^{**}$ is obtained by Equation 4. Specifically, by replacing $g(x^{(i,j)}, \theta)$ in $F(\theta, X, \boldsymbol{\gamma})$ with $\gamma^{*(i,j)}$:

$$\boldsymbol{\gamma}^{**} = \arg\min_{\boldsymbol{\gamma} \in Q} \sum_{(i,j)} -\gamma^{(i,j)} \log \frac{\gamma^{*(i,j)}}{\gamma^{(i,j)}} - (1-\gamma^{(i,j)}) \log \frac{1-\gamma^{*(i,j)}}{1-\gamma^{(i,j)}} \tag{5}$$

We denote the big summation on the right-hand side as $h_1(\boldsymbol{\gamma}^{*}, \boldsymbol{\gamma})$ so $\boldsymbol{\gamma}^{**} = \arg\min_{\boldsymbol{\gamma} \in Q} h_1(\boldsymbol{\gamma}^{*}, \boldsymbol{\gamma})$. We resort to expensive numerical optimizers to find $\boldsymbol{\gamma}^{**}$. Since this is done only once offline, we can afford expensive computation for a more accurate solution. Specifically, to account for the transitivity constraint Q, we add an additional transitivity loss to the objective function. The total amount of transitivity violations of all triplets of tuples is:

$$l_{\text{transitivity}}(\boldsymbol{\gamma}) = \Sigma_{i,j,k} \text{Relu}(\gamma^{(i,j)} \gamma^{(i,k)} - \gamma^{(j,k)}) \tag{6}$$

The constrained solution $\boldsymbol{\gamma}^{**}$ can be obtained by minimizing the following expression with respect to $\boldsymbol{\gamma}$:

$$\text{Loss}(\boldsymbol{\gamma}^{*}, \boldsymbol{\gamma}) = \alpha l_{\text{transitivity}}(\boldsymbol{\gamma}) + h_1(\boldsymbol{\gamma}^{*}, \boldsymbol{\gamma}) \tag{7}$$

where α is a hyperparameter controlling the preference between satisfying the transitivity constraint and minimizing the negative free energy function. We set $\alpha = 100$ as we empirically observed that $\alpha = 100$ ensures the transitivity constraint is satisfied in the final numerical solutions. For each $\boldsymbol{\gamma}^{*}$, we numerically minimize $\text{Loss}(\boldsymbol{\gamma}^{*}, \boldsymbol{\gamma})$ with multiple optimizers including optimizers for nonconvex

optimization [57, 71] using an existing pytorch implementation [39]. Thus, for each $\boldsymbol{\gamma}^*$, we obtain multiple different solutions of $\boldsymbol{\gamma}^{**}$ where each solution is from a different optimizer. We then pick the solution with the smallest loss. Since we expect $\boldsymbol{\gamma}^{**}$ to be close to $\boldsymbol{\gamma}^*$, we always initialize $\boldsymbol{\gamma}^{**}$ as $\boldsymbol{\gamma}^*$ during optimization.

Output Dimension Reduction. Predicting the $\boldsymbol{\gamma}^{**}$ vector requires the model to predict 1024 values at the same time, which would be very difficult. We instead reduce the task to predicting only a single value by exploiting a symmetry property of the task.

Consider the example shown in Figure 3. The naive approach of predicting all 1024 values is shown in Figure 3(a) where the input and output of the model are both a matrix of size 32×32. Imagine we have a model learned to predict a single value in the red cell in Figure 3(a). Originally, the value in this cell is $\gamma^{**}_{0,1}$, which means the model predicts $\gamma^{**}_{0,1}$. Next in Figure 3(b), we swap t_0 with t_2 and swap t_1 with t_3. The rows and columns in the input matrix are swapped accordingly. Now the value in the red cell becomes $\gamma^{**}_{2,3}$ and the model predicts the value $\gamma^{**}_{2,3}$. Similarly, we can make the model predict *any* value in the output matrix by swapping appropriate tuples, except the diagonal values which are known to be 1.

Formally, let h denote the model that takes an input matrix and predicts a single value in the red cell where the original value is $\gamma^{**}_{0,1}$. Let $S^{k,l}_{i,j}(\boldsymbol{\gamma}^*)$ denote swapping t_k with t_i and swapping t_l with t_j. Then, $\forall i, j$ we have $\boldsymbol{\gamma}^{**}_{i,j} = h(S^{0,1}_{i,j}(\boldsymbol{\gamma}^*))$. In this way, we are able to predict any of the 32×32 values in $\boldsymbol{\gamma}^{**}$ with a model h that only predicts a single value. Therefore, we do not need to train a model that predicts the $32 \times 32 = 1024$ values at the same time; we only need to train a model that predicts one value, which is much easier.

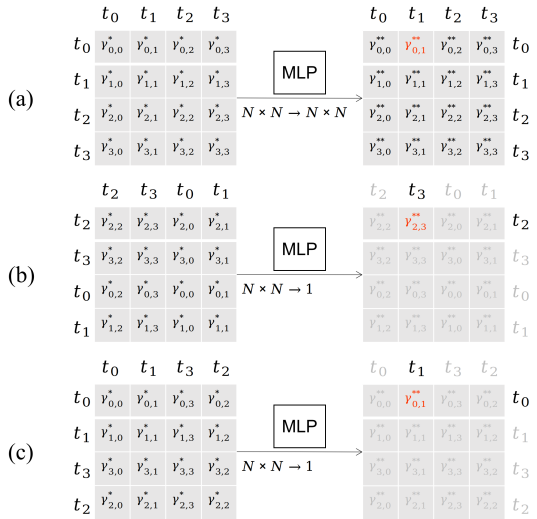


Fig. 3. (a) Naive model. (b) Model with output dimension of 1. (c) Demonstration of invariance to permutation on tuples other than t_0 and t_1.

Swapping-invariant Model Architecture. Consider that in Figure 3(c), when we swap t_2 with t_3, the model h should still predict the value $\gamma^{**}_{0,1}$ and the swapping operation should not impact the prediction. This is a special swapping-invariance property of the problem which we also want to leverage. In general, if we divide the tuples into groups $g_1 = \{t_0, t_1\}$ and $g_2 = \{t_2, \dots\}$, then randomly swapping any tuples any number of times within each group should not change the

prediction of $\gamma^{**}_{0,1}$, as the value in the red cell will always be $\gamma^{**}_{0,1}$ or $\gamma^{**}_{1,0}$, which are equivalent. More formally, we want the following invariance to hold for h:

$$\begin{aligned} h(\boldsymbol{\gamma}^*) =& h(S^{1,0}_{0,1}(\boldsymbol{\gamma}^*)) \\ h(\boldsymbol{\gamma}^*) =& h(S^{k,l}_{i,j}(\boldsymbol{\gamma}^*)), \forall i,j,k,l \text{ that } \{0,1\} \cap \{i,j,k,l\} = \emptyset \end{aligned} \tag{8}$$

We would like to encode this invariance property directly into the model architecture. However, this is very challenging because when swapping the tuples, the corresponding rows and columns in the input matrix change at the same time. Let $\boldsymbol{\gamma}^*_{\text{mat}}$ denote the matrix form of $\boldsymbol{\gamma}^*$. The swapping operation $S^{k,l}_{i,j}(\boldsymbol{\gamma}^*)$ can be also written in a matrix form as $(P^k_i P^l_j)\boldsymbol{\gamma}^*_{\text{mat}}(P^k_i P^l_j)^T$ where P^k_i is the permutation matrix [5] obtained by swapping the ith row and kth row of the identity matrix. Our core idea is to decompose the input matrix into an eigenvector matrix V and eigenvalue matrix using the singular value decomposition $\boldsymbol{\gamma}^*_{\text{mat}} = VWV^T$. Subsequently,

$$\begin{aligned} (P^k_i P^l_j)\boldsymbol{\gamma}^*_{\text{mat}}(P^k_i P^l_j)^T =& (P^k_i P^l_j)VWV^T(P^k_i P^l_j)^T \\ =& (P^k_i P^l_j V)W(P^k_i P^l_j V)^T = V'WV'^T \end{aligned} \tag{9}$$

where $V' = P^k_i P^l_j V$. The V' matrix is obtained by swapping the ith row with the kth row and swapping the jth row with the lth row in the matrix V. This means that swapping the rows and columns in $\boldsymbol{\gamma}^*_{\text{mat}}$ at the same time is equivalent to swapping only the rows in the eigenvector matrix V. Note that since the matrix $\boldsymbol{\gamma}^*_{\text{mat}}$ is symmetric, W and V are guaranteed to be real-valued. Also, while it might be tempting to use the Cholesky decomposition [4] instead (i.e., $\boldsymbol{\gamma}^*_{\text{mat}} = UU^T$), the problem is that U can contain complex numbers which cannot be easily used as inputs for neural networks.

To make the model satisfy the invariance in Equation 8, we decompose the input $\boldsymbol{\gamma}^*_{\text{mat}}$ as W and V, then use W and V as the new input to the model, so that we only need to ensure the model is invariant to row swapping operations on V. Specifically, similar to how we divide the tuples into two groups, we divide the rows in V into two groups $g'_1 = \{\text{1st row, 2nd row}\}$ and $g'_2 = \{\text{3rd row, } \dots\}$, and we need to make sure that randomly swapping/shuffling any rows within each group does not change the model prediction.

Inspired by PointNet [64], we present such a model architecture in Figure 4. The input $\boldsymbol{\gamma}^*$ is first decomposed as V and W. Each row in V is then encoded by a neural network to obtain a row embedding vector. We subsequently take the maximum along every embedding dimension for embedding vectors in each group of rows (g'_1 and g'_2). This gives us one embedding vector for each group. Since the max operation is invariant to any swapping of elements within each group, the model architecture is invariant to row-swapping within each row group (g'_1 or g'_2) of V. The two embedding vectors of the two groups are then concatenated with the diagonal values in W (W is a diagonal matrix) to form one concatenated embedding vector. The concatenated vector finally passes through another neural network which predicts the value $\boldsymbol{\gamma}^{**}_{0,1}$.

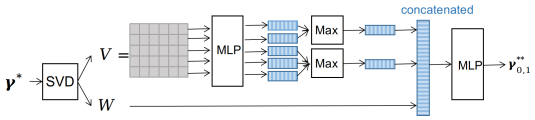


Fig. 4. Transitivity model architecture.

Model Inference. In online inference, we decompose a higher-dimensional input $\boldsymbol{\gamma}^*$ into many *independent vectors* of 1024 dimensions, and we can apply h on each vector. To do so, we form a graph G_M, where nodes are tuples and an edge exists between two nodes only if their matching probability

exceeds 0.5. This is equivalent to a prior approach that relaxes the transitivity constraint Q by only considering transitivity violations that involve at least two predicted matching pairs [80]. The intuition is that transitivity is useful only for tuple pairs predicted to be matches. After identifying the connected components of G_M, we can apply h to tuple pairs in each connected component independently because there is no constraint that involves tuples from two different connected components (otherwise, the two components would have been merged). For each component, if it has fewer than 32 tuples (which covers most cases), we add dummy tuples to form a 1024-dimensional vector (or a 32× 32 matrix) as illustrated in Figure 5(1); if it has more than 32 tuples, then for each edge, we randomly sample 30 neighbors of the two nodes on the edge as illustrated in Figure 5(2); The random sampling is repeated ten times and we take the averaged prediction for that edge after applying h to each sample. Each connected component with a size smaller than 32 will be processed by h only once, so the involved edges will be processed only once. For a connected component with a size greater than 32, each edge will be processed 10 times as we take 10 random samples of its neighbors. Therefore, the time complexity is $O(N)$ where N is the number of edges in the graph (which is the number of tuple pairs in the candidate set) and the time complexity of the overall SIMPLE-EM algorithm is still $O(M_I N \log(N))$. The space complexity of SIMPLE-EM is $O(N)$.

Computational complexity. We summarize the computational complexity in all cases. For two-table EM, when one table is duplicate-free, the time complexity is $O(M_I N \log(N))$ where M_I is the number of iterations (we empirically found $M_I < 10$ suffices) and N is the size of the candidate set; when two tables are duplicate-free, the time complexity is $O(M_I N(\log(N) + \min(N_l, N_r))$ where N_l and N_r are the number of tuples in the left and right table; when no table is duplicate-free, transitivity is not used and the time complexity is $O(M_I N \log(N))$. For single-table EM, the time complexity is $O(M_I N \log(N))$. In all cases, the space complexity is $O(N)$.

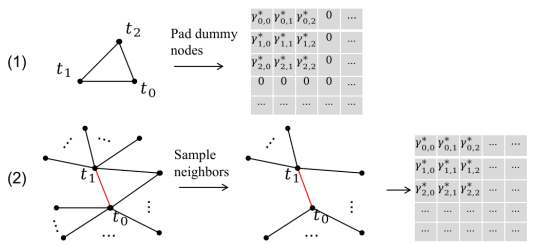


Fig. 5. (1) A small connected component with a size ≤ 32. (2) A large connected component with a size > 32. The matrix on the right is the input for the ML model.

5 EXPERIMENTS

We evaluate our proposed method along five dimensions:

- *Overall Performance.* How does the overall performance of our method (SIMPLE-EM) compare to other existing methods?
- *Transitivity.* How does our proposed method of handling transitivity compare to prior methods?
- *Data shift.* How do traditional manual labeling and weak supervision behave in case of data shift?
- *Sensitivity Analysis.* How sensitive is our method to different LFs?
- *Truth inference on general tasks.* How does the general form of our method (SIMPLE) work on general weak supervision tasks?

5.1 Experimental Setup

Hardware and Platform. We use a machine with a 2.20GHz Intel Xeon(R) Gold 5120 CPU, a K80 GPU and with 96GB 2666MHz RAM. In all experiments for all involved methods, we use GPU whenever possible and we use multi-core parallelization whenever possible.

Datasets. We adopt commonly used real-world two-table and one-table benchmark datasets from the EM literature, such as those maintained by the Leipzig DB Group [1, 45], the Magellan project [25] and the Alaska benchmark [21]. The statistics of the datasets are shown in Table 1. The Monitor and Camera datasets are single-table datasets, and all other datasets are two-table datasets. Note that three datasets (IMDB-Rotten Tomatoes, Yellow Pages-Yelp, and Amazon-Barnes Noble) from the Magellan Repository [25] only contain partial ground-truth (i.e., only a subset of matching pairs and a subset of non-matching pairs are provided). All other datasets have complete ground-truth. Also note that different records in the Alaska benchmark datasets (Monitor and Camera) can have different attributes [21]; in these cases, we keep the 10 most common attributes and fill with NA for records missing these attributes. We design a default blocking strategy for each dataset based on its most informative attributes (e.g., title) using the overlap of tokens. Specifically, we use the *OverlapBlocker* from the py_entitymatching package [3]; the candidate set size and blocking recall are shown in Table 1. The same blocking is used for all baseline methods.

Table 1. Benchmark EM datasets. "-" for N_{Non} denotes that all other tuple pairs are non-matches as the complete set of matches is provided for the dataset.

Dataset	**# tuples** L , R	# matches, # nonmatches, # unlabeled N_M, N_{Non}, N_U	**# attr**	**candset size , recall**
Fodors-Zagats (FZ)	533 , 331	112 , - , 0	7	2915 , 1.0
DBLP-ACM (DA)	2616 , 2294	2224 , - , 0	4	46456 , 0.998
DBLP-Scholar (DS)	2616 , 64263	5347 , - , 0	4	135327 , 0.913
Abt-Buy (AB)	1082 , 1093	1098 , - , 0	4	164072 , 0.995
Amazon-Google (AG)	1363 , 3226	1300 , - , 0	4	42413 , 0.944
Walmart-Amazon (WA)	2554 , 22074	1154 , - , 0	9	68265 , 0.879
IMDB-Rotten Tomatoes (IR)	2960 , 3093	169 , 230 , 63399	10	63798 , 1.0
Yellow Pages-Yelp (YY)	11840 , 5223	131 , 271 , 4876	6	5278 , 1.0
Amazon-Barnes Noble (ABN)	9836 , 9958	233 , 143 , 67769	11	68145 , 0.884
Monitor (M)	16663	26921 , - , 0	10	99230 , 0.953
Camera (C)	29788	314315 , - , 0	10	1101318 , 0.979

Algorithms Evaluated. We first compare our SIMPLE-EM method to five state-of-the-art labeling models (or truth inference methods), which are selected based on prior benchmarks [86, 88].

- *Majority Vote (MV):* For each tuple pair, the labeling model's prediction is the most common label given by the labeling functions.
- *Dawid and Skene's Method (D&S):* This method models each LF's confusion matrix with respect to the ground-truth and optimizes the parameters with the Expectation-Maximization algorithm [26].
- *Enhanced Bayesian Classifier Combination (EBCC):* This method models the joint distribution of LFs with matrix decomposition to reduce the number of parameters [52]. This is the state-of-the-art variant of the Bayesian Classifier Combination based methods.
- *Snorkel (SN):* This is the labeling model used by the Snorkel system [66]. We use the latest open-source implementation [9] which performs truth inference with a matrix completion model [67].
- *Flying Squid (FS):* This is an efficient alternative to the PGM model. It provides a closed-form solution for a triangulated PGM under some assumptions, obviating the need for an iterative EM algorithm or stochastic gradient descent [32].

We further compare to EM solutions designed to require no or less label data including the state-of-the-art unsupervised EM solution, one pre-trained language model based solution, and two active learning based solutions:

• *ZeroER (ZE):* This is the state-of-the-art unsupervised entity matching solution [80]. ZeroER tailors the Gaussian Mixture Model to EM by considering several EM-specific properties. ZeroER uses Magellan [43] to do feature engineering. We use the official implementation [19] that supports both two-table and one-table EM.
• *Ditto:* This is the state-of-the-art entity matching system based on pre-trained language models [51]. Ditto casts the entity matching task as a sequence pair classification task and works by fine-tuning a pre-trained language model for the task. Since the model can capture various semantic meanings of textual data, Ditto is expected to require fewer labeled examples.
• *Active learning based Random forest (AL-RF):* We use the implementation from the most widely used active learning library modAL [23] and use the default query strategy (uncertainty sampling). Active learning requires feature engineering, so we use Magellan [44] to automatically generate features. We use the default strategy to handle class imbalance.
• *Active learning based Random forest with SMOTE (AL-RF-S):* This is a variant of AL-RF that uses SMOTE [17] to handle class imbalance.

Setups for Various Algorithms. The candidate set size after blocking on Monitor and Camera is still huge, so we take a subsample on these two datasets. To not lose any matches in the candidate set, we sample by keeping the tuple pairs where at least one of the two tuples has matches. The candidate set size and recall in Table 1 are obtained after subsampling. For all methods that need a seed, the reported results are the average results of five runs. For methods (e.g., Snorkel) that require a class weight parameter to handle class imbalance, we obtain the class weights by counting the number of matches and non-matches from the Majority Vote method. For active learning and Ditto, we evaluate the performance on a hold out test set which includes 20% of the data.
Performance Metric. We use F1 score as our performance metric since EM is a binary classification task with unbalanced classes.

5.2 Labeling Function Development

Though we consider LF development as an orthogonal task and focus on accurately combining a given set of LFs, we provide the details of how we obtain the LFs for EM datasets as there is no existing open-source LFs. We use an existing tool, the Panda IDE [84], that provides a visual interface to develop LFs efficiently for each dataset. Our developed LFs are available at [8].

We developed LFs for all datasets in Table 1 in the following order: FZ, DA, DS, AB, AG, M, C, WA, IR, YY, and ABN. When writing LFs for one dataset, we may reuse LFs from different attributes of the same dataset or LFs from a prior dataset. To properly measure the effort spent, we report the number of new LFs that require significant effort on each dataset. Specifically, an LF that requires more than 15 seconds is counted as a new LF, while an LF that is obtained in less than 15 seconds by simply modifying the attribute or distance function of an existing LF does not count as a new LF. On each dataset, the number of new LFs is smaller than the total number of LFs because we reuse LFs from prior datasets or across multiple attributes of the same dataset and these "duplicate" LFs are not counted as new LFs.

The statistics of new LFs and total time spent on all datasets are shown in Table 2. The number of LFs ranges from 8 to 16 for the datasets. However, most of the LFs are obtained rapidly by changing the attribute or distance function of existing LFs. Typically, one only needs to develop a few new LFs with some actual effort for a new dataset; the required time ranges from 10 to 50 minutes.

Table 2. Time spent for developing LFs

	FZ	DA	DS	AB	AG	WA	IR	YY	ABN	M	C
# of LFs	12	13	16	13	13	14	8	10	13	10	11
# of new LFs	5	6	4	4	3	2	1	1	2	3	2
time spent, minutes	30	50	45	30	20	15	10	10	15	25	15

5.3 Overall Performance

Labeling Performance. The performance results for weak/un-supervised methods are shown in Table 3. Our method outperforms all baseline methods significantly, achieving the highest F1 score on 9 out of the 11 datasets, and only under-performing EBCC on the Camera (C) dataset and D&S on the IR dataset by less than 1%. Snorkel is the best-performing weak supervision baseline on most datasets, and our model achieves 10% higher F1 score on average across all datasets. Snorkel performs poorly on the AG dataset; one possible reason for this is that the assumptions Snorkel makes (e.g., LFs are conditionally independent) are violated on AG. All methods perform badly on the WA dataset as the dataset is very dirty.

The unsupervised method ZeroER does not work as well as weak supervision methods. This is because unsupervised methods do not use any supervision signals, so they typically only work well on simple datasets like Fodors-Zagats (FZ).

Table 3. F-1 scores for weak/un-supervsied methods

		Weak Supervision					Unsupervised
	SIMPLE-EM	MV	D&S	EBCC	FS	SN	ZE
FZ	**0.996**	0.848	0.973	0.978	0.644	0.942	0.992
DA	**0.991**	0.726	0.339	0.238	0.324	0.958	0.957
DS	**0.911**	0.908	0.896	0.824	0.421	0.904	0.863
AB	**0.906**	0.628	0.686	0.327	0.689	0.776	0.520
AG	**0.555**	0.439	0.426	0.369	0.217	0.199	0.484
WA	**0.499**	0.397	0.332	0.395	0.085	0.363	0.400
IR	0.989	0.985	**0.997**	0.911	0.982	0.958	0.968
YY	**0.969**	0.968	0.952	0.960	0.956	0.953	0.684
ABN	**0.897**	0.834	0.792	0.821	0.240	0.809	0.839
M	**0.887**	0.780	0.708	0.737	0.665	0.812	0.325
C	0.872	0.791	0.791	**0.884**	0.865	0.817	0.477
Avg.	**0.861**	0.755	0.717	0.677	0.553	0.772	0.683

Comparison to Ditto. We compare our method to Ditto, a method based on pre-trained language models that is expected to require less data. We use the implementation from the official GitHub repository and use the default configurations [58]. For some datasets, the official repository also provides pre-split training, validation and test sets that are small subsamples of the candidate set [51]. We first confirm that we are able to get comparable results with the original paper using the provided training, validation and test sets. However, the provided training, validation and test sets are only small subsamples of the candidate set; for example, the provided Walmart-Amazon dataset only includes about 10000 pairs, which is just 15% of our candidate set size. Therefore, to evaluate the performance of Ditto in a more realistic setting and also to ensure the setting of Ditto is comparable to our method, in our experiment we use all data in the candidate set (with ground-truth labels) and randomly split the data into training, validation, and test sets by a ratio of 3:1:1. We report the results in Table 4.

Table 4. Comparison to Ditto (F-1 score)

	FZ	DA	DS	AB	AG	WA	IR	YY	ABN	M	C
SIMPLE-EM	0.996	0.991	0.911	0.906	0.555	0.499	0.989	0.969	0.897	0.887	0.872
Ditto	0.951	0.967	0.933	0.283	0.275	0.262	0.716	0.861	0.708	0.844	0.627

Ditto is better than our method only on the Dblp-Scholar (DS) dataset. Ditto likely falls short of our method for the following reasons: (1) Ditto is sensitive to the creation of training/validation/test sets. For example, on the Abt-Buy dataset, Ditto gets a F1 score of 0.821 on the provided training/validation/test sets in the GitHub repository but only gets an F1 score of around 0.283 on random splits (we got similar results on several splits). (2) Since Ditto is based on pretrained language models, Ditto is expected to have advantages on datasets with many text attributes and may not work well on datasets with numerical features and categorical features.

Comparison to Active Learning. Active learning is also a common technique to obtain labeled data. We compare our method to two active learning methods (AL-RF and AL-RF-S). The results are shown in Table 5. Note that in Table 5, for each dataset, we report the best result between the two active learning methods (the performance of the two methods is similar across datasets, so we do not report them separately). Human time is estimated by assuming each label takes three seconds. On six out of eight datasets, active learning is not able to match the performance of our method even after querying for labels on all data points (which is equivalent to a random forest classifier trained using all labels). Even on the three datasets (DS, WA and C) where active learning matches the performance of our method, active learning requires several hundreds or thousands of labels. The required human time can be as much as 200 minutes, which is significantly more than the required human time for our method (shown in Table 2).

Active learning likely does not work as well as our method for the following reasons: (1) Weak supervision holistically incorporates (weak) information of all data points to infer the ground-truth labels, while active learning infers the decision boundary based on the set of most uncertain data points selected on its query strategy which might not be reliable. Furthermore, on difficult datasets where the decision boundary is complicated, active learning would still need to select many data points. (2) Our method considers the transitivity property of EM which provides additional signals.

Table 5. Comparison to active learning. Note we report the best result of the two active learning methods (AL-RF and AL-RF-S) on each dataset. "-" denotes that active learning is not able to match our method's performance. Human time is estimated by assuming each label takes three seconds. Note that the three datasets (IR, YY, ABN) with only partial ground-truth are not included as active learning may query data points not in the ground-truth.

		AL matches SIMPLE			AL queries all labels	
	SIMPLE-EM	# of labels	% of labels	human time (min)	F1	# of labels
FZ	0.996	-	-	-	0.985	2332
DA	0.991	-	-	-	0.981	37165
DS	0.911	460	0.4%	23	0.938	108262
AB	0.906	-	-	-	0.510	131258
AG	0.555	-	-	-	0.539	33931
WA	0.499	350	0.5%	17.5	0.695	3150
M	0.887	-	-	-	0.848	79384
C	0.872	4310	0.5%	215	0.949	881055

Running Time. The memory requirements of the methods are comparable except that Majority Vote requires much less memory and Ditto requires much more memory. The running times of all methods are shown Table 6. Note the table only shows machine time, and human time for labeling is not included. For weak supervision methods, the running time includes the time for applying the LFs to all tuple pairs in the candidate set to obtain the weak labels and the time for inferring the ground-truth labels. The unsupervised method ZeroER (ZE) and active learning method (AL-RF) require feature engineering to be done. We use Magellan [44] to automatically perform feature engineering for the two methods following prior work [80]. For these two methods (ZE and AL-RF), the running time includes the time for feature engineering and the time for training and inference.

For Ditto, we use the default configuration from the official implementation [58] and the running time includes the time for preprocessing the textual data (e.g., tokenization), the time for data augmentation, the time for text summarization, and the time for training and inference [51].

The running time of SIMPLE-EM is greater than other weak supervision methods, as performing cross validation to select the hyperparameters for the random forest classifier can be relatively expensive. However, this can be alleviated by using more CPUs because cross validation can be easily parallelized. Overall, the unsupervised method ZeroER and the active learning method AL-RF have much higher running time than SIMPLE-EM because these two methods require feature engineering, which is expensive. In addition, AL-RF updates the model when querying each new data point which is expensive. Ditto has the longest running time as it involves multiple expensive steps like tokenizing the text data, performing data augmentation and text summarization, and training a deep learning model. Note that for the three datasets (IR, YY and ABN) with partial labeled data, we only use the small labeled subset of data for training Ditto, so the running time of Ditto is extremely small on these three datasets.

Table 6. Running time (minutes) for all methods. Note only machine time is included even for active learning (AL-RF).

	Weak Supervision						Unsupervised	Supervised	
	SIMPLE-EM	MV	D&S	EBCC	FS	SN	ZE	AL-RF	Ditto
FZ	0.6	0.1	0.2	0.1	0.1	0.1	1.5	3.7	4.4
DA	3.7	0.6	1.8	1.2	0.7	0.6	16.3	18.1	59.2
DS	15.3	3.1	7.7	3.5	3.1	3.3	52.4	57.2	113.7
AB	17.9	3.7	8.4	4.7	3.9	3.7	71.8	81.5	140.3
AG	7.9	0.8	3.3	1.1	0.9	0.8	22.4	26.9	127.4
WA	6.8	1.4	3.6	1.7	1.5	1.4	179.2	194.1	214.8
IR	6.5	0.9	2.7	1.3	1.1	0.9	28.2	-	1.4
YY	1.7	0.3	0.4	0.3	0.3	0.3	4.5	-	0.8
ABN	12.8	0.6	11.6	1.1	0.9	0.6	30.3	-	1.2
M	5.5	0.7	5.1	0.9	0.8	0.7	128.1	133.4	151.3
C	46.5	6.3	42.2	15.1	7.4	6.3	197.9	216.0	248.7
Avg.	11.4	1.7	7.9	2.9	1.9	1.7	66.6	91.4	96.7

End Model EM Performance. Our generated labels can be used to train any downstream models, such as existing supervised ML methods. We use DeepMatcher [59] as an example downstream model to demonstrate the effectiveness our generated labels for training an end model. Using the open-source implementation [11], we compare the DeepMatcher model trained with our generated labels with the DeepMatcher model trained with ground-truth labels. For each dataset, we divide the tuple pairs after blocking into training, validation, and test sets by a ratio of 3:1:1. **(1) SIMPLE-EM Labels:** We use our generated labels for the training and validation set to train a model and use the ground-truth labels for the test set to evaluate the trained model. **(2) Ground-Truth Labels:** We train another end model with ground-truth labels in the training and validation set. To measure the labeling effort saved by SIMPLE-EM, we gradually increase the training set size to match the performance of the end model trained on our generated labels. We report the number of ground-truth labels required to match the performance of the end model trained on our labels and the number of ground-truth labels (when F1 score is converged) where adding more ground-truth labels does not improve F1 score by a meaningful amount.

Overall, when using ground-truth labels, at least several thousand labels are required for the end model to match its performance when trained on the generated labels from SIMPLE-EM. In addition, the F1 score of end model trained on SIMPLE-EM labels is comparable (though on average 3.1% worse) to the converged F1 of the end model trained on sufficient amount of ground-truth labels.

Table 7. DeepMatcher trained on SIMPLE-EM labels vs DeepMatcher trained on Ground-Truth (GT) labels. The "*" symbol on the converged # GT labels denotes that the dataset only has partial ground-truth labels and the number precedes "*" is 80% of the ground-truth labels (the other 20% are used as the test set).

	FZ	DA	DS	AB	AG	WA	IR	YY	ABN	M	C	Avg.
F1 of DeepMatcher on SIMPLE-EM labels	0.979	0.978	0.926	0.673	0.741	0.450	1.0	1.0	0.898	0.956	0.929	0.866
# GT labels to match above F1	2333	37170	48284	164078	11015	10947	-	-	265	7586	8866	32282
Converged F1	0.979	0.978	0.956	0.692	0.845	0.631	0.964	0.923	0.903	0.999	0.999	0.897
# GT labels	2333	37170	120710	218770	55075	16421	320*	322*	285*	75860	122200	59042

We observed that the performance of the end model trained on SIMPLE-EM's labels can sometimes be better than the original labeling performance. On the AG dataset in particular, the end model F1 score is 0.741, which is about 20% better than the original labels (0.555). The reason is that the end model can incorporate additional information (e.g., textual features) that is different from the information used in LFs. This phenomenon is also observed in multiple prior works on weak supervision [24, 77, 82].

On the AG dataset, the end model trained with ground-truth labels (which have F1 score of 1) gets an F1 score of at most 0.845, which is only 10% better than the end model trained on SIMPLE-EM labels. This is in spite of the fact that the SIMPLE-EM labels have a 50% worse F1 score than the ground-truth labels. Our takeaway is that training end models with noisy and weak supervision often suffices, while accurate supervision by an expensive process of labeling individual data points may not be necessary. This confirms prior empirical findings [66, 78] and theoretical derivations [68, 72].

5.4 Handling Transitivity Constraint

We conduct experiments to compare different ways of handling transitivity. We compare the following methods:

- *No trans:* This is to ignore transitivity and directly use our base labeling model (SIMPLE) in Section 3.
- *SIMPLE-EM:* This is our proposed method in Section 4.
- *ZeroER Trans:* We replace our component of handling transitivity with the one in ZeroER [80]. ZeroER uses a projection based heuristic to solve Equation 4 to enforce transitivity.
- *Postprocessing:* This is the traditional way of handling transitivity in a postprocessing step. Specifically, the *No trans* method (SIMPLE) is used to obtain a prediction, then a postprocessing step is done on the prediction to enforce transitivity. The way to perform postprocessing differs for two-table datasets and single-table datasets. (1) On single-table datasets, we have the matching probability for all pairs, so it is possible to use clustering methods in ER literature. We use the method adopted by dedupe [35] (hierarchical clustering with centroid linkage [61]) and we also use the implementation in dedupe [35]. (2) On two-table datasets, we do not have the matching probabilities for tuple pairs from the same table. In order to do postprocessing, following prior work [80], we assume that the left and right table are duplicate-free and the matching probability of any tuple pair from the same table to be 0 for all two-table datasets. For example, we have three tuples t_{l_1}, t_{l_2}, and t_{r_1} where t_{l_1} and t_{l_2} come from the left table and t_{r_1} is from the right table. We assume the matching probability of t_{l_1} and t_{l_2} to be 0, i.e., $\gamma^{(l_1,l_2)} = 0$. When the transitivity constraint is violated, we keep the cross-table tuple pair with higher probability as a match. For example, when $\gamma^{(l_1,r_1)} = 0.8$ and $\gamma^{(l_2,r_1)} = 0.9$, the transitivity constraint is violated and we keep (t_{l_2}, t_{r_1}) as a match and discard the tuple pair (t_{l_1}, t_{r_1}).

The results are shown in Table 8. Overall, our method of handling transitivity works the best on all datasets and improves F1 score by about 9% on average. The method of handling transitivity in ZeroER is not robust across datasets as it employs a greedy algorithm to correct each triplets of

Table 8. Different methods to handle transitivity.

	FZ	DA	DS	AB	AG	WA	IR	YY	ABN	M	C	Avg.
No Trans	0.978	0.765	**0.911**	0.697	**0.555**	0.493	**0.988**	**0.969**	0.884	0.781	0.832	0.805
SIMPLE-EM	**0.996**	**0.991**	**0.911**	**0.906**	**0.555**	**0.499**	**0.989**	**0.969**	**0.897**	**0.887**	**0.872**	**0.861**
ZeroER Trans	0.993	**0.991**	0.880	0.794	0.413	**0.499**	0.988	0.670	**0.897**	0.694	0.142	0.724
Postprocess	0.990	0.979	0.625	0.343	0.486	0.494	0.985	0.670	0.894	0.610	0.633	0.701

tuple pairs that violates transitivity [80]. In contrast, our method holistically considers all tuple pairs when enforcing transitivity. Postprocessing also does not work well and is even worse than *No trans*, e.g. on AB. This is because postprocessing can introduce spurious matches or remove true matches [15, 80]. Intuitively, postprocessing separates the clustering stage from the inference stage and therefore each stage uses less information, while our method considers clustering as an integral part of the inference stage and thus exploits all information holistically.

5.5 Controlled Study of Transitivity

5.5.1 A Survey on Transitivity Violations in real-world datasets. We investigate the frequency of transitivity violations in the ground-truth of real-world datasets and examine the cause of the violations.

We first introduce how we detect transitivity violations in the ground-truth. For single-table datasets, three tuples t_i, t_j and t_k constitute a violation if in the ground-truth (t_i, t_j) and (t_i, t_k) are matches but (t_j, t_k) is a non-match. For two-table datasets, since only ground-truth labels of the cross-table tuple pairs are provided [25], we can only identify transitivity violations with the following method: Let $t_{l,i}$ and $t_{l,j}$ denote two tuples from the left table; Let $t_{r,i}$ and $t_{r,j}$ denote two tuples from the right table; The transitivity property is violated when $(t_{l,i}, t_{r,i})$, $(t_{l,j}, t_{r,i})$ and $(t_{l,i}, t_{r,j})$ are matches and $(t_{l,j}, t_{r,j})$ is not a match. To see this, when $(t_{l,i}, t_{r,i})$ and $(t_{l,j}, t_{r,i})$ are matches, the tuples $t_{l,i}$ and $t_{l,j}$ are the same entity by the transitivity property, so if $(t_{l,i}, t_{r,j})$ is a match $(t_{l,j}, t_{r,j})$ must be a match, otherwise the transitivity property is violated.

In the 11 datasets we used in our experiments, only two datasets (DS and WA) contain transitivity violations in the ground-truth (on which SIMPLE-EM still outperforms other methods, see Table 3.). We further inspected all 30 datasets in the Magellan repo [25], and found that only 6 datasets contain transitivity violations in the ground. For these 6 datasets, on average only 4% of the labeled tuple pairs in the ground-truth are involved in a violation. This validates a belief commonly-held in the literature that transitivity is mostly satisfied in real-world scenarios [16, 46, 80].

To investigate the causes of transitivity violations, we manually inspect a random sample of the detected violations. We found that in 40% of the cases, the violation of transitivity is due to incorrect matching pairs in the ground-truth. For example, (t_i, t_j) is a match and (t_i, t_k) should not be a match, but the ground-truth includes both (t_i, t_j) and (t_i, t_k) as matches. The other 60% of the cases are caused by incomplete matching labels in the ground-truth. For example, both (t_i, t_j) and (t_i, t_k) are actual matches and are included in the ground-truth, (t_j, t_k) is also a match (based on a manual inspection) but is not included as a match in the ground-truth.

5.5.2 Varying the Amount of Transitivity Violations. We perform a controlled experiment to evaluate how different methods behave when we vary the amount of transitivity violations in the ground-truth. From our survey on 30 real-world datasets, transitivity is violated when the ground-truth is incomplete or incorrect. Therefore, we can perform this experiment by corrupting the ground-truth labels to make the set of matching pairs incomplete or incorrect.

The Camera and Monitor datasets have the highest number of labeled tuple pairs involved in transitivity constraints in the ground-truth, while other datasets have fewer such tuple pairs (e.g., the ground-truth matching pairs in the FZ dataset are all one-to-one mappings so no tuple pairs are involved in transitivity). In addition, the Camera and Monitor datasets are "perfectly" labeled in

the sense that initially there are no transitivity violations in the ground-truth. Therefore, we use the Camera and Monitor datasets for this experiment. We inject transitivity violations with the following steps: We first randomly select a tuple t_i that has matches in the ground-truth. Next, by a probability of 60% we randomly mark one true matching pair that involves t_i as a non-match, and in the remaining 40% of cases we randomly choose a non-matching pair that involves t_i and mark it as a spurious match. These probabilities are selected based on our survey on real-world datasets in Section 5.5.1. We repeat the two steps xN_{gt} times, where N_{gt} is the total number of matches in the original ground-truth; and x controls the amount of violations we introduce (from 0 to 0.5 with a step size of 0.1). We compare with the two best performing baseline methods from Table 3: Snorkel and Majority Vote. We report the averaged score of all methods over the two datasets in Table 9. As x increases, the performance of all methods decreases because transitivity violations are achieved by corrupting the ground-truth labels. However, SIMPLE-EM always performs better than the two other baseline methods.

Table 9. F1-score of top methods when varying the amount of transitivity violations in the ground-truth (x).

x	0	0.1	0.2	0.3	0.4	0.5
SIMPLE-EM	0.880	0.841	0.802	0.764	0.726	0.697
SN	0.815	0.777	0.740	0.705	0.670	0.636
MV	0.786	0.750	0.715	0.681	0.648	0.616

5.6 Adaptation to Data Shift

Data shift is a challenging problem in many real-world applications [65]. In this section, we study the behavior of LFs and traditional manual labeling during data shift.

In our datasets in Table 1, we have two pairs of datasets with the same schema: (DA, DS) and (AB, AG). In addition, we have (AB, WA) where the attributes of WA is a super-set of the attributes in AB. We construct a new dataset WA' from WA using the subset attributes appearing also in AB so that the constructed dataset WA' has the same schema with AB. To simulate data shift, we consider the following source-target data shift: DA-DS (shift from DA to DS), AB-AG (shift from AB to AG), AB-WA' (shift from AB to WA'). We use the following settings for LFs and manual labeling:

• *LFs.* When developing LFs in Section 5.2, we reused many LFs from prior datasets. For each source-target dataset pair, we count the total number of LFs N_1' in the target dataset. Since some of the LFs are reused from the source dataset, we count the number of newly developed LFs N_2' for the target dataset. We report $\frac{N_1'-N_2'}{N_1'}$ as the amount of saved effort for labeling the target dataset.

• *Manual labeling.* For each source-target dataset pair, we consider the task of learning on the target dataset. First, we ignore the source dataset and only consider the target dataset, and we use the active learning method. We count the number of labels N_1 queried by active learning when it reaches the performance of LFs (or its peak performance if it cannot match LFs). Second, we add all labeled data from the source dataset to the training set of active learning on the target dataset. Since the source and target dataset has the same schema, the features generated by Magellan [44] are the same, so we can directly use the labeled data from the source dataset to train a model for the target dataset. We then count the number of additional labels N_2 from the target dataset queried by active learning when it reaches the performance of LFs (or its peak performance if it cannot match LFs) on the target dataset. We report $\frac{N_1-N_2}{N_1}$ as the amount of saved effort for labeling the target dataset by using the manual labels in the source dataset.

The results are shown in Table 10. In manual labeling, the labeled data on the source dataset is not always helpful for the target dataset. For example, on the AB-AG datasets and the AB-WA'

datasets, the saved labeling effort is negative. This means, when using labeled data from the source, one actually needs to label more data on the target dataset to achieve the same performance as in the case where one simply ignores the source dataset. This is understandable as the definition of being a match can be different on a different dataset. Even when the labeled data from the source dataset is helpful (e.g., on the DA-DS datasets), the saved labeling effort is still significantly smaller than that for using LFs. In contrast, using LFs is more flexible and one can easily adapt to the new definition of being a match on the new dataset by reusing a subset of original LFs or by adding more LFs.

In addition, we highlight that when the source and target datasets have different schemas (features are different), there is no way for manual labeling to reuse existing labels from the source dataset; However, for LFs, as long as there are some common or similar attributes, one can easily reuse the LFs written on these attributes.

Table 10. Saved labeling effort on the target dataset for LFs and manual labeling under data shift.

data shift	manual labeling	LFs
DA-DS	31.5%	62.5%
AB-AG	-23.2%	63.6%
AB-WA'	-9%	73.3%

5.7 Sensitivity Analysis

We analyze labeling model performance under varying sets of LFs.

LF Randomization. Some LFs use threshold values to assign labels (e.g., the name_overlap LF in Figure 1 compares the score variable to two different threshold values). To assess LF sensitivity, we generate a new set of LFs from the original LFs by randomly tweaking the threshold values (if any) within a range around the original thresholds. Additionally, we also take a random sample of the full set of LFs to see how methods perform with fewer LFs.

Results. Table 11 shows the sensitivity of our method and baseline methods to changes in the LFs. Our method outperforms all baselines in all scenarios and generally shows less of a degradation in performance than other methods at 80% and 60% LF sampling. When the sampled proportion of LFs decreases, the performance improvement of our method over baseline methods decreases. This is because when there are fewer LFs, the benefit of carefully combining them decreases, as also reported in prior work [66].

Table 11. Sensitivity to LFs. RT denotes randomized thresholds. $x\%$ denotes sampling $x\%$ of the original set of LFs. The original number of LFs for each dataset can be found in Table 2. The scores are F1-scores averaged over all datasets.

	Original	RT+100%	RT+80%	RT+60%	RT+40%
SIMPLE-EM	0.861	0.856	0.831	0.766	0.570
MV	0.755	0.718	0.599	0.554	0.537
D&S	0.717	0.672	0.595	0.474	0.397
EBCC	0.677	0.632	0.638	0.533	0.504
SN	0.772	0.698	0.674	0.623	0.554
FS	0.553	0.486	0.460	0.447	0.445

5.8 Truth Inference on General Tasks

We evaluate our proposed method (SIMPLE) from Section 3 on general weak supervision tasks to verify that our proposed method works beyond EM. We consider all ten binary classification datasets in the WRENCH weak supervision benchmark repository [10, 86]. For all datasets, we use

Table 12. Performance on the wrench benchmark [86]. "F1" denotes F1-score and "acc" denotes accuracy score.

Datasets	# of LFs	metric	SIMPLE	MV	D&S	EBCC	FS	SN
basketball	4	F1	0.171	**0.181**	0.171	0.171	0.171	0.144
commercial	4	F1	0.837	0.846	0.778	0.775	0.763	**0.878**
tennis	6	F1	0.844	**0.847**	**0.847**	**0.847**	**0.847**	0.841
yelp	8	acc	**0.744**	0.722	0.683	0.696	0.709	0.696
imdb	8	acc	**0.750**	0.737	0.744	0.744	0.744	**0.750**
spouse	9	F1	**0.517**	0.492	0.343	0.343	0.505	0.455
youtube	10	acc	**0.916**	0.853	0.452	0.452	0.845	0.847
cdr	33	F1	**0.713**	0.672	0.001	0.087	0.104	0.666
sms	73	F1	0.825	0.838	0.650	0	0	**0.840**
census	83	F1	**0.527**	0.330	0.001	0	0.209	0.445
Avg.	-	-	**0.684**	0.652	0.467	0.412	0.490	0.656

the provided LFs and performance metric from the benchmark [10, 86]. The results are shown in Table 12.

Our proposed method outperforms the best baseline on average by 3%. This is significant considering that the best prior work only outperforms Majority Vote by 0.4%. It's quite surprising that our method obtains such good performance in spite of its simplicity. In contrast, existing methods typically use very complicated models (e.g., graphical models [32] and matrix completion models [67]).

6 RELATED WORK

Entity Matching. Supervised learning algorithms achieve the best results on entity matching [43, 50, 59, 60, 87]. However, they require large amounts of labeled data. To reduce training data size in supervised EM, active learning based approaches selectively label useful tuple pairs, but need to have human annotators involved in the ML training process [13, 74]. Transfer learning is adopted to reuse information from existing datasets or pretrained language models [50], but is not robust when the target dataset is very different from source datasets or language models as we have shown in our experiments. Different from existing work, our work programmatically generates labels by adapting weak supervision (that has been successfully applied to label generation for general ML tasks [66, 69, 78]) to EM. Our work offers a new method to perform entity matching when no labeled data is available.

Truth Inference. The existing truth inference methods are typically designed for general tasks [49, 52]. Our SIMPLE-EM method is specifically designed for EM by incorporating the transitivity property of EM and achieves better performance than general truth inference methods. In addition, existing methods are mostly hand-crafted, complicated models with various assumptions to implicitly restrict the hypothesis space to avoid trivial solutions [49, 52, 66, 69]. In contrast, our method is based on a generic classifier and makes no assumptions. Our method restricts the hypothesis space in an explicit data-driven fashion through cross validation. The two distinctions of our method enabled our method to achieve better results on both general tasks and EM tasks.

7 CONCLUSION

In this work, we present a labeling model to generate high-quality EM labels by combining the predictions of different labeling functions in a weak supervision setting. We first propose a simple and powerful general labeling model for general weak supervision classification tasks. We then tailor the method to the task of EM by ensuring the predicted labels satisfy the transitivity property of EM. Finally, we experimentally validate that our general labeling model works well on ten weak supervision datasets and find that the tailored version for EM is significantly better than existing approaches across several diverse datasets.

REFERENCES

[1] [n.d.]. Benchmark datasets for entity resolution. https://dbs.uni-leipzig.de/research/projects/object_matching/benchmark_datasets_for_entity_resolution.

[2] [n.d.]. Competera Product Matching for Price Comparison. https://competera.net/solutions/by-need/product-matching.

[3] 2021. Blocking — py_entitymatching 0.4.0 documentation. http://anhaidgroup.github.io/py_entitymatching/v0.4.0/user_manual/api/blocking.html [Online; accessed 6. Jul. 2022].

[4] 2021. Cholesky decomposition - Wikipedia. https://en.wikipedia.org/w/index.php?title=Cholesky_decomposition&oldid=1059421881 [Online; accessed 21. Jan. 2022].

[5] 2021. Permutation matrix - Wikipedia. https://en.wikipedia.org/w/index.php?title=Permutation_matrix&oldid=1059174802 [Online; accessed 11. Dec. 2021].

[6] 2021. scipy.sparse.csgraph.min_weight_full_bipartite_matching — SciPy v1.7.1 Manual. https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.min_weight_full_bipartite_matching.html#scipy.sparse.csgraph.min_weight_full_bipartite_matching [Online; accessed 9. Dec. 2021].

[7] 2022. Ground Truth Inference for Weakly Supervised Entity Matching (technical report). https://figshare.com/s/6d57cabada80b1e3d42d.

[8] 2022. SIMPLE: data and code. https://figshare.com/s/60a4b1595827bb44d5ae

[9] 2022. snorkel. https://github.com/snorkel-team/snorkel [Online; accessed 23. Jan. 2022].

[10] 2022. wrench. https://github.com/JieyuZ2/wrench [Online; accessed 23. Feb. 2022].

[11] anhaidgroup. 2022. deepmatcher. https://github.com/anhaidgroup/deepmatcher [Online; accessed 7. Jan. 2022].

[12] Anonymous. 2023. Learning Hyper Label Model for Programmatic Weak Supervision. In *Submitted to The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=aCQt_BrkSjC under review.

[13] Arvind Arasu, Michaela Götz, and Raghav Kaushik. 2010. On active learning of record matching packages. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*. 783–794.

[14] Eric Arazo, Diego Ortego, Paul Albert, Noel E O'Connor, and Kevin McGuinness. 2020. Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.

[15] Jurian Baas, Mehdi Dastani, and Ad Feelders. 2021. Exploiting transitivity constraints for entity matching in knowledge graphs. *arXiv preprint arXiv:2104.12589* (2021).

[16] Christoph Böhm, Gerard De Melo, Felix Naumann, and Gerhard Weikum. 2012. LINDA: distributed web-of-data-scale entity matching. In *Proceedings of the 21st ACM international conference on Information and knowledge management*. 2104–2108.

[17] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research* 16 (2002), 321–357.

[18] Peter Christen. 2012. Data matching: concepts and techniques for record linkage, entity resolution, and duplicate detection. Springer.

[19] chu-data lab. 2022. zeroer. https://github.com/chu-data-lab/zeroer [Online; accessed 10. Jul. 2022].

[20] Contributors to Wikimedia projects. 2022. Variational Bayesian methods - Wikipedia. https://en.wikipedia.org/w/index.php?title=Variational_Bayesian_methods&oldid=1071116594 [Online; accessed 25. Mar. 2022].

[21] Valter Crescenzi, Andrea De Angelis, Donatella Firmani, Maurizio Mazzei, Paolo Merialdo, Federico Piai, and Divesh Srivastava. 2021. Alaska: A Flexible Benchmark for Data Integration Tasks. *arXiv preprint arXiv:2101.11259* (2021).

[22] Hong Cui, Jingjing Zhang, Chunfeng Cui, and Qinyu Chen. 2016. Solving large-scale assignment problems by Kuhn-Munkres algorithm. In *2nd Int. Conf. Advances Mech. Eng. Ind. Inform.(AMEII 2016)*.

[23] Tivadar Danka and Peter Horvath. [n.d.]. modAL: A modular active learning framework for Python. ([n. d.]). https://github.com/modAL-python/modAL available on arXiv at https://arxiv.org/abs/1805.00979.

[24] Nilaksh Das, Sanya Chaba, Renzhi Wu, Sakshi Gandhi, Duen Horng Chau, and Xu Chu. 2020. Goggles: Automatic image labeling with affinity coding. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1717–1732.

[25] Sanjib Das, AnHai Doan, Paul Suganthan G. C., Chaitanya Gokhale, Pradap Konda, Yash Govind, and Derek Paulsen. [n.d.]. The Magellan Data Repository. https://sites.google.com/site/anhaidgroup/useful-stuff/the-magellan-data-repository.

[26] Alexander Philip Dawid and Allan M Skene. 1979. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 28, 1 (1979), 20–28.

[27] Gianluca Demartini, Djellel Eddine Difallah, and Philippe Cudré-Mauroux. 2012. Zencrowd: leveraging probabilistic reasoning and crowdsourcing techniques for large-scale entity linking. In *Proceedings of the 21st international conference on World Wide Web*. 469–478.

[28] Xin Luna Dong. 2019. Building a Broad Knowledge Graph for Products. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 25. https://doi.org/10.1109/ICDE.2019.00010

[29] Xin Luna Dong and Theodoros Rekatsinas. 2018. Data integration and machine learning: A natural synergy. In *Proceedings of the 2018 international conference on management of data*. 1645–1650.

[30] Ahmed K Elmagarmid, Panagiotis G Ipeirotis, and Vassilios S Verykios. 2007. Duplicate Record Detection: A Survey. *IEEETKDE* 19, 1 (2007), 1–16.

[31] Jason A Fries, Paroma Varma, Vincent S Chen, Ke Xiao, Heliodoro Tejeda, Priyanka Saha, Jared Dunnmon, Henry Chubb, Shiraz Maskatia, Madalina Fiterau, et al. 2019. Weakly supervised classification of aortic valve malformations using unlabeled cardiac MRI sequences. *Nature communications* 10, 1 (2019), 1–10.

[32] Daniel Fu, Mayee Chen, Frederic Sala, Sarah Hooper, Kayvon Fatahalian, and Christopher Ré. 2020. Fast and three-rious: Speeding up weak supervision with triplet methods. In *International Conference on Machine Learning*. PMLR, 3280–3291.

[33] Huiji Gao, Geoffrey Barbier, and Rebecca Goolsby. 2011. Harnessing the crowdsourcing power of social media for disaster relief. *IEEE Intelligent Systems* 26, 3 (2011), 10–14.

[34] Lise Getoor and Ashwin Machanavajjhala. 2012. Entity resolution: theory, practice and open challenges. *PVLDB* 5, 12 (2012), 2018–2019.

[35] Forest Gregg and Derek Eder. 2022. Dedupe. *https://github.com/dedupeio/dedupe* (2022).

[36] Thomas N Herzog, Fritz J Scheuren, and William E Winkler. 2007. *Data Quality and Record Linkage Techniques*. Springer Science & Business Media.

[37] Petr Hruby, Timothy Duff, Anton Leykin, and Tomas Pajdla. 2022. Learning to Solve Hard Minimal Problems. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5532–5542.

[38] Shahana Ibrahim and Xiao Fu. 2021. Crowdsourcing via Annotator Co-occurrence Imputation and Provable Symmetric Nonnegative Matrix Factorization. In *International Conference on Machine Learning*. PMLR, 4544–4554.

[39] jettify. 2021. pytorch-optimizer. https://github.com/jettify/pytorch-optimizer [Online; accessed 10. Dec. 2021].

[40] Roy Jonker and Anton Volgenant. 1987. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing* 38, 4 (1987), 325–340.

[41] David Karger, Sewoong Oh, and Devavrat Shah. 2011. Iterative learning for reliable crowdsourcing systems. *Advances in neural information processing systems* 24 (2011).

[42] Daphne Koller and Nir Friedman. 2009. *Probabilistic graphical models: principles and techniques*. MIT press.

[43] Pradap Konda, Sanjib Das, Paul Suganthan GC, AnHai Doan, Adel Ardalan, Jeffrey R Ballard, Han Li, Fatemah Panahi, Haojun Zhang, Jeff Naughton, et al. 2016. Magellan: Toward building entity matching management systems. *Proceedings of the VLDB Endowment* 9, 12 (2016), 1197–1208.

[44] Pradap Venkatramanan Konda. 2018. *Magellan: Toward building entity matching management systems*. The University of Wisconsin-Madison.

[45] Hanna Köpcke, Andreas Thor, and Erhard Rahm. 2010. Evaluation of entity resolution approaches on real-world match problems. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 484–493.

[46] Simon Lacoste-Julien, Konstantina Palla, Alex Davies, Gjergji Kasneci, Thore Graepel, and Zoubin Ghahramani. 2013. Sigma: Simple greedy matching for aligning large knowledge bases. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 572–580.

[47] Dong-Hyun Lee et al. 2013. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *Workshop on challenges in representation learning, ICML*, Vol. 3. 896.

[48] Peng Li, Xiang Cheng, Xu Chu, Yeye He, and Surajit Chaudhuri. 2021. Auto-FuzzyJoin: Auto-Program Fuzzy Similarity Joins Without Labeled Examples. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 1064–1076. https://doi.org/10.1145/3448016.3452824

[49] Qi Li, Yaliang Li, Jing Gao, Bo Zhao, Wei Fan, and Jiawei Han. 2014. Resolving conflicts in heterogeneous data by truth discovery and source reliability estimation. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 1187–1198.

[50] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep Entity Matching with Pre-Trained Language Models. *Proc. VLDB Endow.* 14, 1 (Sept. 2020), 50–60. https://doi.org/10.14778/3421424.3421431

[51] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep entity matching with pre-trained language models. *arXiv preprint arXiv:2004.00584* (2020).

[52] Yuan Li, Benjamin Rubinstein, and Trevor Cohn. 2019. Exploiting worker correlation for label aggregation in crowdsourcing. In *International Conference on Machine Learning*. 3886–3895.

[53] Yinghao Li, Pranav Shetty, Lucas Liu, Chao Zhang, and Le Song. 2021. BERTifying the Hidden Markov Model for Multi-Source Weakly Supervised Named Entity Recognition. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 6178–6190.

[54] Pierre Lison, Jeremy Barnes, Aliaksandr Hubin, and Samia Touileb. 2020. Named Entity Recognition without Labelled Data: A Weak Supervision Approach. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 1518–1533.

[55] Qiang Liu, Jian Peng, and Alexander T Ihler. 2012. Variational inference for crowdsourcing. *Advances in neural information processing systems* 25 (2012).

[56] Gilles Louppe. 2014. Understanding random forests: From theory to practice. *arXiv preprint arXiv:1407.7502* (2014).

[57] Xuezhe Ma. 2020. Apollo: An adaptive parameter-wise diagonal quasi-newton method for nonconvex stochastic optimization. *arXiv preprint arXiv:2009.13586* (2020).

[58] megagonlabs. 2022. ditto. https://github.com/megagonlabs/ditto [Online; accessed 6. Jul. 2022].

[59] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep Learning for Entity Matching: A Design Space Exploration. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 19–34. https://doi.org/10.1145/3183713.3196926

[60] Ebraheem Muhammad, Thirumuruganathan Saravanan, Joty Shafiq, Nan Tang, and Ouzzani Mourad. 2018. Distributed Representations of Tuples for Entity Resolution. *Proceedings of the VLDB Endowment* 11, 11 (2018).

[61] Fionn Murtagh and Pedro Contreras. 2012. Algorithms for hierarchical clustering: an overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 2, 1 (2012), 86–97.

[62] Radford M Neal and Geoffrey E Hinton. 1998. A view of the EM algorithm that justifies incremental, sparse, and other variants. In *Learning in graphical models*. Springer, 355–368.

[63] Eniola Olaleye. 2022. WINNING APPROACH ML COMPETITION 2022 - Machine Learning Insights - Medium. *Medium* (Mar 2022). https://medium.com/machine-learning-insights/winning-approach-ml-competition-2022-b89ec512b1bb

[64] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. 2017. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 652–660.

[65] Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D Lawrence. 2008. *Dataset shift in machine learning*. Mit Press.

[66] Alexander Ratner, Stephen H. Bach, Henry R. Ehrenberg, Jason Alan Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proc. VLDB Endow.* 11, 3 (2017), 269–282. https://doi.org/10.14778/3157794.3157797

[67] Alexander Ratner, Braden Hancock, Jared Dunnmon, Frederic Sala, Shreyash Pandey, and Christopher Ré. 2019. Training complex models with multi-task weak supervision. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4763–4771.

[68] Alexander J Ratner, Christopher M De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. 2016. Data programming: Creating large training sets, quickly. *Advances in neural information processing systems* 29 (2016), 3567–3575.

[69] Alexander J. Ratner, Christopher De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. 2016. Data Programming: Creating Large Training Sets, Quickly. In *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain*, Daniel D. Lee, Masashi Sugiyama, Ulrike von Luxburg, Isabelle Guyon, and Roman Garnett (Eds.). 3567–3575. http://papers.nips.cc/paper/6523-data-programming-creating-large-training-sets-quickly

[70] Vikas C Raykar, Shipeng Yu, Linda H Zhao, Gerardo Hermosillo Valadez, Charles Florin, Luca Bogoni, and Linda Moy. 2010. Learning from crowds. *Journal of machine learning research* 11, 4 (2010).

[71] S Reddi, Manzil Zaheer, Devendra Sachan, Satyen Kale, and Sanjiv Kumar. 2018. Adaptive methods for nonconvex optimization. In *Proceeding of 32nd Conference on Neural Information Processing Systems (NIPS 2018)*.

[72] Joshua Robinson, Stefanie Jegelka, and Suvrit Sra. 2020. Strength from weakness: Fast learning using weak supervision. In *International Conference on Machine Learning*. PMLR, 8127–8136.

[73] Salva Rühling Cachay, Benedikt Boecking, and Artur Dubrawski. 2021. End-to-End Weak Supervision. *Advances in Neural Information Processing Systems* 34 (2021).

[74] Sunita Sarawagi and Anuradha Bhamidipaty. 2002. Interactive deduplication using active learning. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 269–278.

[75] Jaeho Shin, Sen Wu, Feiran Wang, Christopher De Sa, Ce Zhang, and Christopher Ré. 2015. Incremental knowledge base construction using deepdive. In *Proceedings of the VLDB Endowment International Conference on Very Large Data Bases*, Vol. 8. NIH Public Access, 1310.

[76] Michael Stonebraker and Ihab F Ilyas. 2018. Data Integration: The Current Status and the Way Forward. *IEEE Data Eng. Bull.* 41, 2 (2018), 3–9.

[77] Paroma Varma and Christopher Ré. 2018. Snuba: Automating weak supervision to label training data. In *Proceedings of the VLDB Endowment. International Conference on Very Large Data Bases*, Vol. 12. NIH Public Access, 223.

[78] Paroma Varma and Christopher Ré. 2018. Snuba: Automating Weak Supervision to Label Training Data. *Proc. VLDB Endow.* 12, 3 (nov 2018), 223–236. https://doi.org/10.14778/3291264.3291268

[79] Matteo Venanzi, John Guiver, Gabriella Kazai, Pushmeet Kohli, and Milad Shokouhi. 2014. Community-based bayesian aggregation models for crowdsourcing. In *Proceedings of the 23rd international conference on World wide web*. 155–164.

[80] Renzhi Wu, Sanya Chaba, Saurabh Sawlani, Xu Chu, and Saravanan Thirumuruganathan. 2020. ZeroER: Entity Resolution using Zero Labeled Examples. *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (May 2020), 1149–1164. https://doi.org/10.1145/3318464.3389743

[81] Renzhi Wu, Shen-En Chen, Jieyu Zhang, and Xu Chu. 2023. Learning Hyper Label Model for Programmatic Weak Supervision. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=aCQt_BrkSjC

[82] Renzhi Wu, Nilaksh Das, Sanya Chaba, Sakshi Gandhi, Duen Horng Chau, and Xu Chu. 2022. A Cluster-then-label Approach for Few-shot Learning with Application to Automatic Image Data Labeling. *ACM Journal of Data and Information Quality (JDIQ)* 14, 3 (2022), 1–23.

[83] Renzhi Wu, Bolin Ding, Xu Chu, Zhewei Wei, Xiening Dai, Tao Guan, and Jingren Zhou. 2021. Learning to Be a Statistician: Learned Estimator for Number of Distinct Values. *Proc. VLDB Endow.* 15, 2 (oct 2021), 272–284. https://doi.org/10.14778/3489496.3489508

[84] Renzhi Wu, Prem Sakala, Peng Li, Xu Chu, and Yeye He. 2021. Demonstration of Panda: A Weakly Supervised Entity Matching System. *Proc. VLDB Endow.* 14, 12 (jul 2021), 2735–2738. https://doi.org/10.14778/3476311.3476332

[85] Sen Wu, Luke Hsiao, Xiao Cheng, Braden Hancock, Theodoros Rekatsinas, Philip Levis, and Christopher Ré. 2018. Fonduer: Knowledge base construction from richly formatted data. In *Proceedings of the 2018 international conference on management of data*. 1301–1316.

[86] Jieyu Zhang, Yue Yu, Yinghao Li, Yujing Wang, Yaming Yang, Mao Yang, and Alexander Ratner. 2021. WRENCH: A Comprehensive Benchmark for Weak Supervision. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=Q9SKS5k8io

[87] Chen Zhao and Yeye He. 2019. Auto-em: End-to-end fuzzy entity-matching using pre-trained deep models and transfer learning. In *The World Wide Web Conference*. 2413–2424.

[88] Yudian Zheng, Guoliang Li, Yuanbing Li, Caihua Shan, and Reynold Cheng. 2017. Truth inference in crowdsourcing: Is the problem solved? *Proceedings of the VLDB Endowment* 10, 5 (2017), 541–552.

Received April 2022; revised July 2022; accepted August 2022